

ゲーム開発初心者のためのHowTo
Pyxelでサクッと
レトロゲーム開発



ゲーム開発初心者のための HowTo pyxel でサクッとレト ロゲーム開発！

Hisa Mail: hisanori.andou@tinycore.co.jp

2019-09-17 版 タイニーコア（株） <https://tinycore.co.jp>
発行

目次

第 1 章	はじめに	1
第 2 章	開発環境を作る	3
2.1	Visual Studio Code のダウンロードとインストール	4
2.2	python3 のダウンロードとインストール	8
2.3	pyxel のダウンロードとインストール	9
2.4	Visual Studio Code で拡張ライブラリを入れる	10
2.5	ゲーム開発用フォルダを用意する	12
第 3 章	開発環境に慣れてみよう	14
3.1	ゲーム開発用フォルダを開く	14
3.2	新規ファイルの作成	15
	ファイルエクスプローラーの表示	15
	ファイルエクスプローラーから新規ファイル作成	16
3.3	ゲーム画面に「hello」を表示	17
第 4 章	ゲーム画面に自分で描いたキャラを登場させる	23
4.1	pyxeleditor の起動	23
4.2	ドット絵を描いてみよう	25
4.3	ドット絵を表示してみよう	28
4.4	ドット絵を動かしてみよう	31
第 5 章	敵キャラ（ばいきんまん）をアンパンチする	36
5.1	簡易もぐらたたきゲームを作る	36
5.2	敵キャラを描く	36
5.3	敵キャラを一定周期で表示させる①（準備編）	38

目次

5.4	敵キャラを一定周期で表示させる②（実装編）	41
5.5	いよいよばいきんまんをやっつけする	46
第6章	アニメーションをやってみよう	52
6.1	アンパンチのアニメーションの作り方（手順）	52
6.2	pyxeditor でアンパンチ画像を作成する	53
6.3	アニメーションをテストしてみる	56
6.4	ついでにばいきんまんの画像とアニメーションを作ってみる	59
6.5	実際のプログラムにアニメーションを組み込んでみる（キャラ クラス）	61
	自キャラクラスの解説	63
	敵キャラクラスの解説	65
6.6	実際のプログラムにアニメーションを組み込んでみる（メイン クラス）	66
第7章	背景を作成する	72
7.1	マップチップの作成	72
	校庭ベースチップを作成	73
	トラックマップチップ作成	73
7.2	マップデータの作成	75
7.3	実際のゲームに組み込んでみる	78
第8章	さいごに（おまけに）	82
8.1	モチーフ替え	82
8.2	その他にやるとよさそうなこと	85
8.3	Tip1：pyxeditor で作成したリソースファイルは分解可能	86
8.4	Tip2：python がない環境でも実行できる実行ファイルを作成する	87
	prxres ファイルを「assets」フォルダに格納する	87
	プログラムを修正する	88
	実行ファイルを作成する	88
8.5	まとめ	89

第 1 章

はじめに

今の世の中、「自分でゲーム作りたい!」という人のために色々な環境が提供されています。ですが、その多くのツールが扱い方自体とても難しく、ゲーム作成方法の学習が大変すぎて途中で「もうだめだー!」となってしまうことがあります。

もし、この書籍を手にとっていただいている方がすでにゲームプログラマーとして優秀なスキルをお持ちはこの本をそっと閉じて、ぜひ Unity などの本格的なゲーム開発の書籍を手にとってゲーム開発をしてください。

この本は

- ゲームを作ってみたいけど、プログラム自体よくわからない
- 過去に Unity とかでゲームを作ろうとして挫折してしまった
- python を初めて学んでみたい

等の初めての方向けの「ゲーム開発本」になっております。

今回紹介するゲーム作成の HowTo は「python 言語で pyxel というレトロゲームエンジンを使い、もぐらたたきゲームを作ってみる」という内容です。

レトロゲームというのはいわゆる「ドット絵キャラ」が登場する「ファミコン」っぽいゲームを指します。「pyxel」というレトロゲームを開発するためのモジュールを使って開発をしていきます。この「pyxel」と利用する言語「python」は学習コストが低く最初に学ぶプログラムとして、とても向いています。

本誌では、

- pyxel でゲームを作成するための環境構築
- ゲーム作成環境に慣れてみる

第1章 はじめに

- Step By Step でゲーム開発をする
 - 自キャラ、敵キャラ作成、あたり判定、マップ作成、アニメーション作成など
- さいごに (Tips とかまとめとか)

の内容でゲーム開発を学ぶことができます。ぜひ最後までお付き合いくださいませ。

第 2 章

開発環境を作る

それでは、まず pyxel でゲームを作るための開発環境を整えてみたいと思います。今回は以下の環境を使います。

- 対象の OS: WINDOWS10(64bit)
- エディタ: Visual Studio Code
- python のバージョン: 3.7
- pyxel のバージョン: 1.2 系

WINDOWS 以外の方で pyxel によるゲーム開発を行いたい方は本誌では環境構築を紹介しておりませんが、Linux、Mac 等の場合、以下 URL を参照していただくと各 OS の開発環境構築手順が紹介されております。

<https://github.com/kitao/pyxel>

それでは WINDOWS10 に、

- Visual Studio Code
- python3.7
- pyxel ライブラリ
- 拡張ライブラリ

をインストール（パソコンに入れること）をしていこうと思います。

2.1 Visual Studio Code のダウンロードとインストール

まず、python のプログラミング、プロジェクトの管理に使うエディタ (IDE ツールとも呼ばれる) として「Visual Studio Code」を使いたいと思います。「Visual Studio Code」は完全無料で利用することができ、さまざまなプラグインやライブラリを追加することで、色々な言語や開発環境をまとめることができます。

Visual Studio Code は以下サイトからインストーラをダウンロードすることができます。

<https://code.visualstudio.com/>

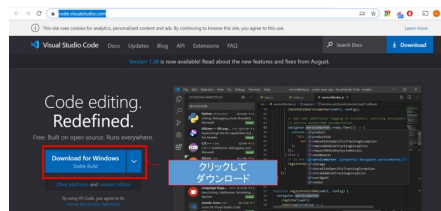


図 2.1: Visual Studio Code の DL

ファイルはそこそこ大きいのでダウンロードに環境によっては1～3分くらいかかると思います。ダウンロードが完了したらダウンロードしたセットアップファイル「VSCodeUserSetup-xxxxx.exe」を実行します。(xxxxx はバージョン番号)

第2章 開発環境を作る

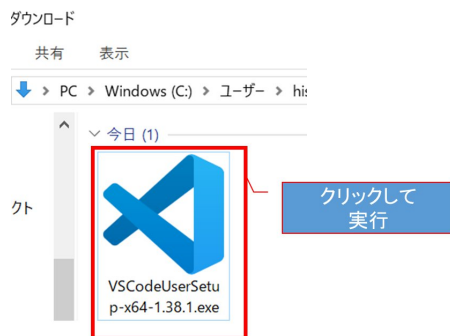


図 2.2: Visual Studio Code のセットアップ①

手順では 64bit 用のセットアップを実行しています。自分のパソコンの OS が 32bit か 64bit か確認してインストールしましょう。

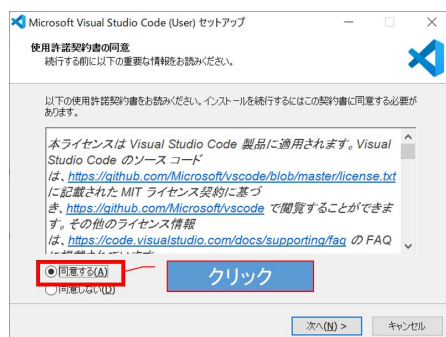


図 2.3: Visual Studio Code のセットアップ②

最初に使用許諾のウィンドウがあるので「同意する」にチェックをいれて「次へ」をクリックします。後は基本、デフォルトのままになります。

第 2 章 開発環境を作る

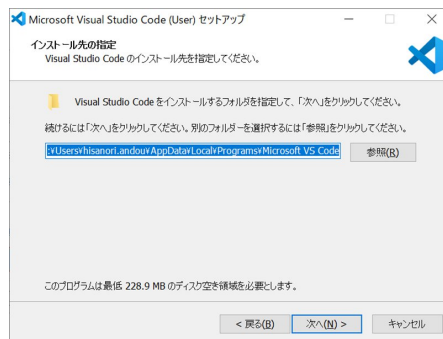


図 2.4: Visual Studio Code のセットアップ③

ここでも特にパスを変更せずにデフォルト設定のまま「次へ」をクリックします。

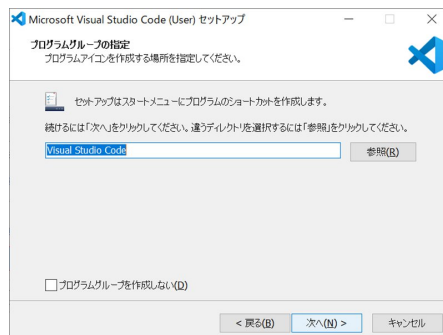


図 2.5: Visual Studio Code のセットアップ④

ここもデフォルト設定のまま「次へ」をクリックします。

第 2 章 開発環境を作る

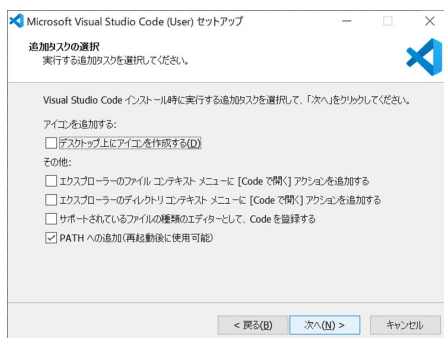


図 2.6: Visual Studio Code のセットアップ⑤

PATH への追加は必須ですが、デフォルトでチェックついているので、そのまま「次へ」をクリックします。

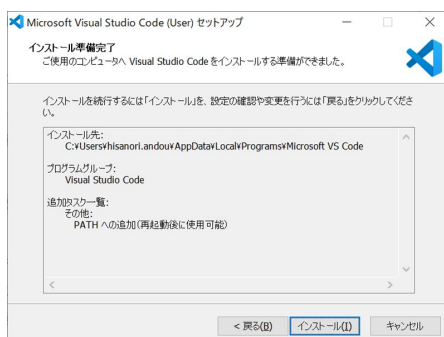


図 2.7: Visual Studio Code のセットアップ⑥

この画面まできたら「インストール」をクリックしてインストール開始します。

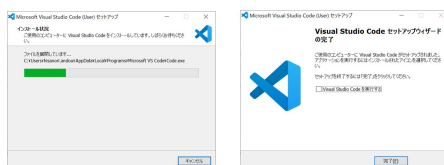


図 2.8: Visual Studio Code のセットアップ⑦

第2章 開発環境を作る

しばらくインストールが完了するまで待ちます。インストールが完了したら「Visual Studio Code を実行する」のチェックを外しておきます。「完了」をクリックしてインストールを終了します。

2.2 python3 のダウンロードとインストール

python3 は以下サイトからインストーラをダウンロードすることができます。

<https://www.python.org/downloads/>

次図のように画面上のボタンをクリックしてください。ダウンロードが開始されます。

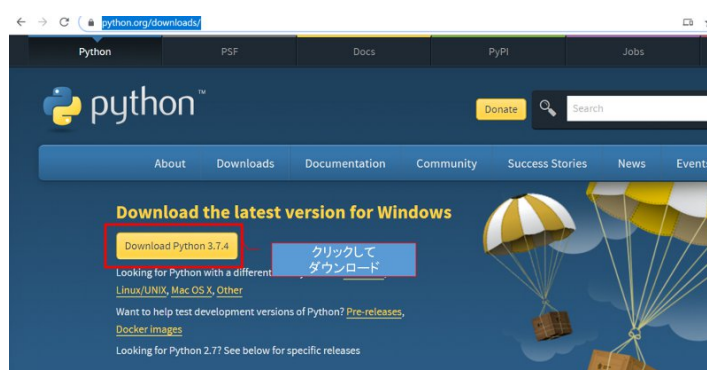


図 2.9: python3 のダウンロード

ダウンロードが完了したら、セットアップファイル「setup.exe」を実行してください。



図 2.10: python3 のセットアップ①

第2章 開発環境を作る

最初に「パスを追加するか」というメッセージが表示されている場合、必ず「チェック」を入れてパスを追加するようにしてください。（※ここでパスをきちんと通す設定をしないとリソース編集ツール「pyxeleditor」を起動することができないので、注意してください！！）

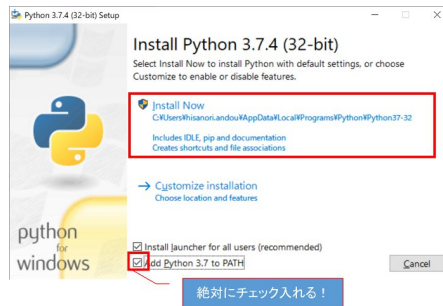


図 2.11: python3 のセットアップ②

インストールが完了するまでしばらく待ちましょう。なお、python3 は OS が 64bit でも 32bit モードでインストールされます。

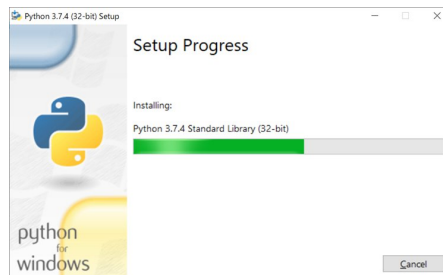


図 2.12: python3 のセットアップ③

2.3 pyxel のダウンロードとインストール

python3 のインストールに成功した場合、一度パソコンを電源を落として、再起動させてください。パソコン再起動後、python3 のインストール時に設定した python3 へのパスが有効になり、python3 の各種コマンド（pip など）が使えるようになります。

第2章 開発環境を作る

次に、pyxel ライブラリを「pip」コマンドでインストールします。

次図のようにパソコンの左下の「WINDOWS マーク」を右クリックし「ファイル名を指定して実行」で「cmd」を入力して ENTER を押下します。

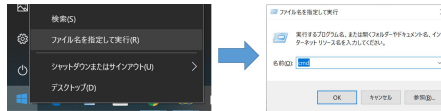


図 2.13: pyxel のインストール①

次に黒い画面（＝コマンドプロンプト画面）が表示されたら以下コマンドを実行します。

```
pip install -U pyxel
```

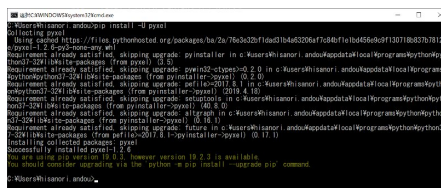


図 2.14: pyxel のインストール②

特にエラーなくインストールが完了したらこの画面を「x」をクリックして閉じます。

2.4 Visual Studio Code で拡張ライブラリを入れる

Visual Studio Code で python を扱うための拡張ライブラリを追加します。この python 用拡張ライブラリを入れると python のスペルミスやプログラムミスを自動でチェックしてくれるようになり、便利です。

右下の「WINDOWS マーク」をクリック→「Visual Studio Code」を起動してください。

第 2 章 開発環境を作る

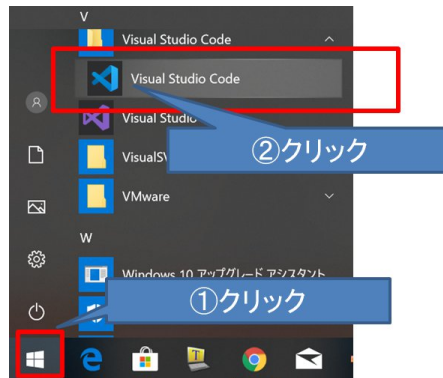


図 2.15: 拡張ライブラリインストール①

Visual Studio Code が起動したら次図のようにウインドウ左にある「拡張」アイコンをクリックします。

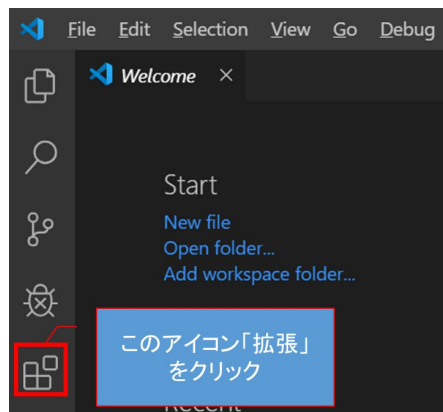


図 2.16: 拡張ライブラリインストール②

検索ワードに「python」入力して python のライブラリをインストールします。

第2章 開発環境を作る

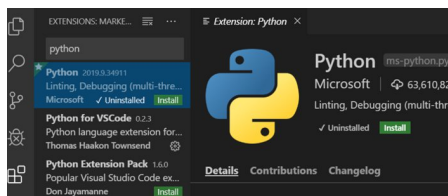


図 2.17: 拡張ライブラリインストール③

「python」の「install」をクリックします。

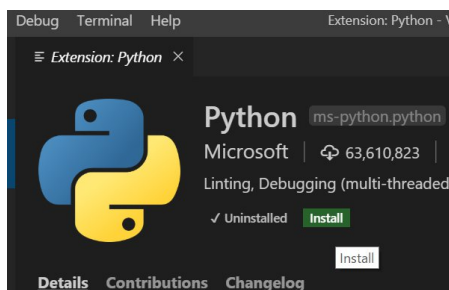


図 2.18: 拡張ライブラリインストール④

インストールが完了したら一度、Visual Studio Code を終了させておきます。
(※設定を有効化するため) これで python が Visual Studio Code で使えるようになりました。

2.5 ゲーム開発用フォルダを用意する

次に、Visual Studio Code で python ゲームをデバッグなどをしやすくするために開発用のフォルダを用意します。

とりあえず以下の図のように C ドライブに新規フォルダ「pyxelGames」を作成して、その中に「.vscode」というフォルダを作ります。

第2章 開発環境を作る

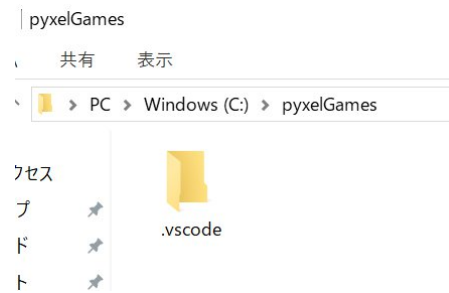


図 2.19: 開発用フォルダ

先ほど作成した「.vscode」フォルダの中に「launch.json」というファイル名で以下の内容をテキストデータを作成して配置します。このファイルがあると、python のデバッグなどが非常に簡単にできるようになります。

リスト 2.1: launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File",
      "type": "python",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal"
    }
  ]
}
```

第 3 章

開発環境に慣れてみよう

ここまでの準備ができましたら、ゲームプログラム環境の構築は完了です。使い方の説明に移りたいと思います。

ゲーム作成で最初につまずくポイントは、開発環境の使い方がよくわからない！ があると思います。Unity など、ゲーム開発のためのメニューや機能が多いツールの場合、まず「使い方」がわからないせいで高確率でつまずきます。そこで、まずは Visual Studio Code と python の開発の環境に慣れてみましょう。

3.1 ゲーム開発用フォルダを開く

Visual Studio Code を起動してください。起動したら「File」→「Open Folder」選択してください。

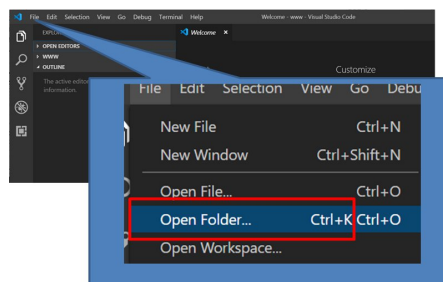


図 3.1: 開発用フォルダを開く①

フォルダ選択ダイアログが表示されたら、先ほど作成した「pyxelGames」フォ

第3章 開発環境に慣れてみよう

ルダを選択します。

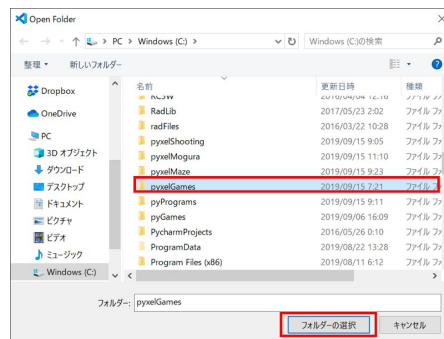


図 3.2: 開発用フォルダを開く②

3.2 新規ファイルの作成

それでは、最初のプログラム「hello world」を作る準備のためにまずは

- 新規ファイルの作成

方法を学びます。新規作成するにはいくつか方法がありますが、今回は一番わかりやすい（使いやすい）方法を説明します。

一番使いやすい新規ファイル作成は「ファイルエクスプローラー」から実行するのがおすすめです。

ファイルエクスプローラーの表示

Visual Studio Code は色々プログラムのデバッグや拡張ライブラリのインストールとかやっているとウインドウ左側に表示されるはずの「ファイルエクスプローラー（選択したフォルダ内のファイルを表示するところ）」がよく隠れてしまいます。

第3章 開発環境に慣れてみよう

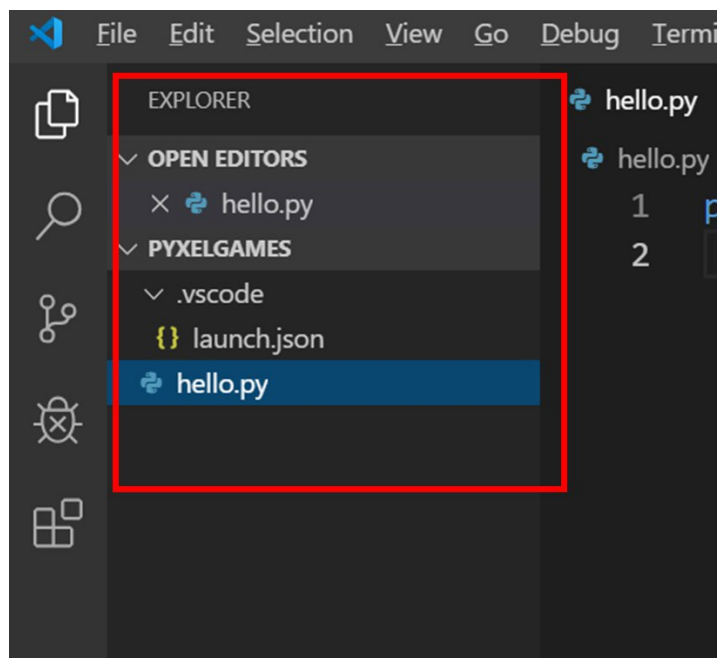


図 3.3: ファイルエクスプローラー

もし「ファイルエクスプローラー」が隠れてしまったらウインドウ左にある
 アイコンをクリックしてファイルエクスプローラーを表示させましょ
う。このアイコンをクリックするたびにファイルエクスプローラーを表示⇄非表
示に切り替えすることができます。

とりあえず、新規ファイル作成するためのファイルエクスプローラーを表示し
ておいてください。

ファイルエクスプローラーから新規ファイル作成

ファイルエクスプローラー内の開いたフォルダ名（※今回は「PYXEL-
GAMES」）の部分を見てください。

第3章 開発環境に慣れてみよう

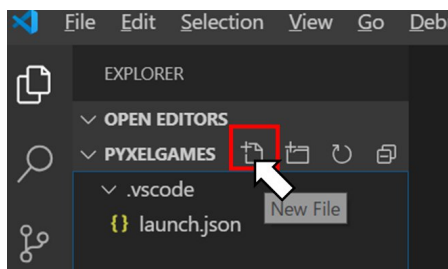


図 3.4: ファイル新規作成①

このフォルダ名の右にマウスカーソルを移動させると、「」という新規ファイル作成アイコンが表示されます。このアイコンをクリックして、新規ファイルを作成します。

次図のようにファイル名を入力する箇所に「hello.py」と入力し ENTER で決定します。

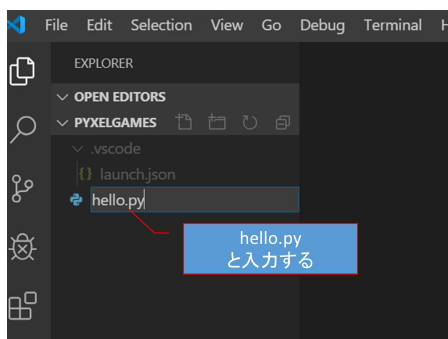


図 3.5: ファイル新規作成②

これでファイルの新規作成は完了です。続いてプログラムを書いていきます。

3.3 ゲーム画面に「hello」を表示

それでは、最初はゲームの画面に「hello」を文字を表示してみましょう。新規作成した「hello.py」ファイルに以下のプログラムを記述します。

リスト 3.1: hello.py

第3章 開発環境に慣れてみよう

```
1: import pyxel
2: WIDTH = 60
3: HEIGHT = 50
4: pyxel.init(WIDTH, HEIGHT)
5: pyxel.cls(1)
6: pyxel.text(0, 2, "Hello", 8)
7: pyxel.show()
```

次に作成したプログラムを実行します。

キーボードの F5 キーを押すか、Visual Studio Code のメニューの「Debug」→「Start Debugging」をクリックするとプログラムをデバッグ実行することができます。

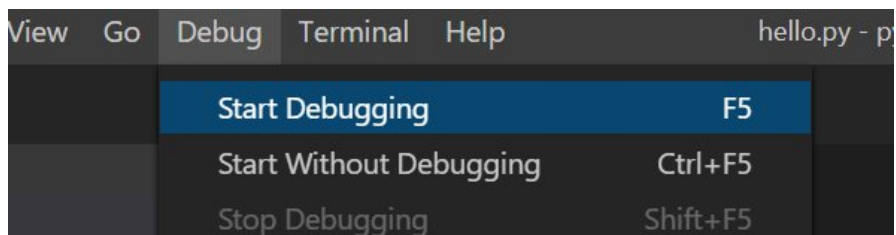


図 3.6: デバッグ実行 [scale=0.5]

プログラムの起動に成功したら次図のようなウィンドウが画面に表示されるはずです。

第3章 開発環境に慣れてみよう



図 3.7: はじめてのプログラム

hello という文字をゲーム画面に表示することができました！（なお、pyxel では日本語を表示することはできません。英数文字のみです）

それでは、プログラムを解説していきましょう。

```
1: import pyxel
```

ゲームエンジンである pyxel を使うための命令です。このコードを書いたあとは pyxel クラスを使うことができます。

```
2: WIDTH = 60  
3: HEIGHT = 50
```

ここで指定するのは、画面の高さ (HEIGHT) と幅 (WIDTH) です。ただし、この数字はゲーム画面のウィンドウサイズではなく「ドットの数」になります。つまり、この設定だと「横に 60 ドット、縦に 50 ドット」のゲーム画面を作る、という意味になります。指定可能な画面ドット数は縦、横ともに最大 256 ドット

第3章 開発環境に慣れてみよう

となります。試しに最大のサイズで実行してみましょう。プログラムは次のように書き換えます。

```
2: WIDTH = 256  
3: HEIGHT = 256
```

現在開いているゲーム画面を一度閉じて、再度デバッグ実行します。

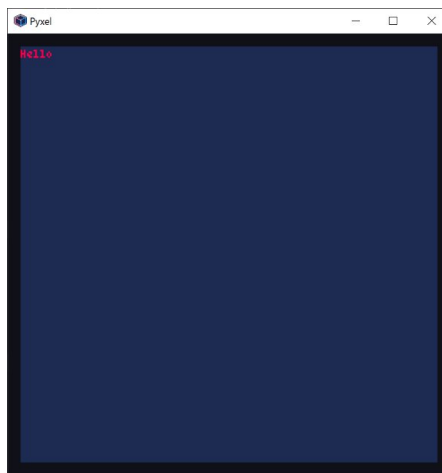


図 3.8: 画面サイズ変更

このようにゲーム画面のドットが縦横ともに 256 に増えたことがわかります。

```
4: pyxel.init(WIDTH, HEIGHT)
```

この「init」という命令でゲーム画面の縦横のドット数を設定します。

```
5: pyxel.cls(1)
```


第3章 開発環境に慣れてみよう

cls 関数は指定した色番号で画面全体を塗りつぶしする、という命令です。色番号には 0～15 の数字を指定することができ、色と数字の関係は次の図のようになっています。

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

図 3.9: 色番号対応

- 0 : 黒
- 1 : ネイビーブルー
- 2 : むらさき（濃）
- 3 : 緑
- 4 : 茶色
- 5 : グレー（濃）
- 6 : グレー（薄）
- 7 : 白
- 8 : 赤
- 9 : だいだい
- 10 : 黄色
- 11 : 黄緑
- 12 : 水色
- 13 : むらさき（薄）
- 14 : ピンク
- 15 : ペールオレンジ

プログラム上では、数字は色番号を指定して利用するので、このリストを手元においておくと便利です。

第3章 開発環境に慣れてみよう

```
6: pyxel.text(0, 2, "Hello", 8)
```

この「text」という命令は「指定した座標に指定した色で文字列を表示する」というものです。

```
pyxel.text(  
    文字の表示位置 X,  
    文字の表示位置 Y,  
    "表示したい文字列",  
    文字色)
```

となります。

```
7: pyxel.show()
```

この「show」という命令でゲームウインドウを表示状態にします。(※実際のゲーム作成ではこの「show」という関数は利用しませんが、このようにちょっとした動作を確認するだけの場合には便利な関数です)

第 4 章

ゲーム画面に自分で描いたキャラを登場させる

ただ hello がでるだけでは何も面白くありません。ゲーム画面に自分で描いたキャラクターを表示させて、動かしてみたいと思います。手順としては

- pyxeleditor というツールを使ってドット絵キャラを作成
- python でキャラを表示するプログラムを作成
- 実行する

という感じです。

まずは、「pyxeleditor」の使い方（お絵描き）を学びましょう。

4.1 pyxeleditor の起動

Visual Studio Code からターミナル画面経由で pyxeleditor を起動することができます。Visual Studio Code の上部メニューの「Terminal」→「New Terminal」を実行してください。

第4章 ゲーム画面に自分で描いたキャラを登場させる

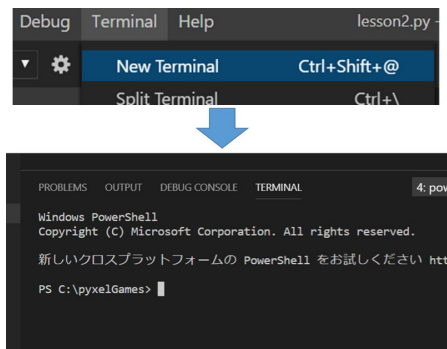


図 4.1: ターミナル起動

すると、ウインドウ内にコマンドを打つためのターミナル画面が表示されます。

```
pyxeleditor mychara
```

と入力し、ENTER を押します。

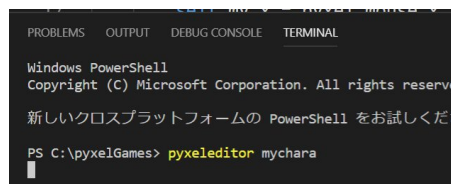


図 4.2: pyxeleditor 起動コマンド

この「mychara」とはこれから作成するリソースファイル（＝画像データを保存するファイル）の名前になります。このコマンドでこの「pyxelGames」フォルダの中に「mychara.pyxres」というリソースファイルが生成されます。（正確には保存ボタンを押したあとにファイルが生成されます）

起動に成功すると、次図のようなウインドウが表示されます。

第4章 ゲーム画面に自分で描いたキャラを登場させる

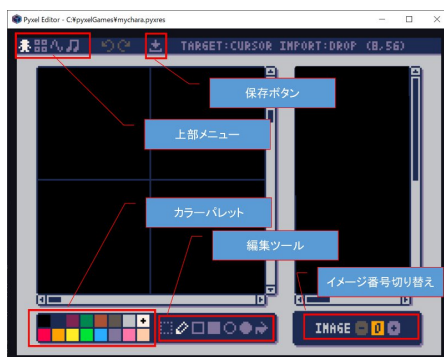


図 4.3: pyxeleditor

- 上部メニュー：キャラ作成やマップ作成を切り替えられます（詳細は後述）
- 保存ボタン：保存ボタンクリックで編集中的数据をファイルに保存します。
- カラーパレット：編集ツールで選択したペンや領域選択時の色を指定します。
- 編集ツール：画像を編集します（詳細は後述）
- イメージ番号切り替え（詳細は後述）

この pyxeleditor の機能を使ってドット絵キャラを作成していきます。

4.2 ドット絵を描いてみよう

それでは、実際に pyxeleditor を使ってキャラクターの絵を描いてみましょう。キャラクターは「8x8」か「16x16」ドットのサイズで描くようにしましょう。

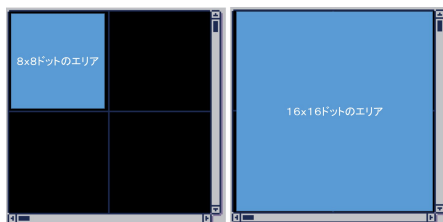


図 4.4: サイズ別描画領域

第4章 ゲーム画面に自分で描いたキャラを登場させる

最初は「16x16」でキャラクターを描いてみます。このサイズの場合、表示画面いっぱいには絵を描くことができます。それでは、ツールの使い方を少し説明します。



- 選択ツールです。マウスで領域をドラッグし、「Ctrl キー +C キー」で領域をコピーし、「Ctrl キー +V キー」でペーストすることができます。同じ絵を何枚もコピーしたい場合に便利です。(アニメーション画面を作るときなど)



- ペンツールです。カラーパレットで選択した色で1ドットの点や線（ドラッグ）を描くことができます。



- 四角枠ツールです。カラーパレットで選択した色で1ドットの四角い枠を描くことができます。



- 四角塗りつぶしツールです。カラーパレットで選択した色で四角形を描くことができます。



- 丸枠ツールです。カラーパレットで選択した色で1ドットの丸枠を描くことができます。



- 丸塗りつぶしツールです。カラーパレットで選択した色で丸を描くことができます。



- 塗りつぶしツールです。カラーパレットで選択した色でクリックした範囲内 (同じ色の部分) を塗りつぶしすることができます。(ペイントツールと同じ)

カラーパレットで色を変えながらペンツールでキャラクターを描いていきま

第4章 ゲーム画面に自分で描いたキャラを登場させる

す。ウインドウ右下が「」のように「0」になっていることを確認します。これは「イメージ番号」と呼ばれる数字で0～3まで合計4枚のイメージデータを扱うことができます。今回はイメージ番号0のイメージデータにキャラクターを描いていきます。

オリジナルキャラクターが難しい場合は、アンパンマンとかの既存キャラクターを描いてみると描きやすいかもしれません。



図 4.5: キャラを描く

こんな感じでアンパンマンを描いてみました。最後に保存を忘れずに実行して

おきます。保存はウインドウ上部にある保存ボタン「」をクリックして保存することができます。

画面上部メニューについて説明をしておきます。

-  今回使ったキャラクター作成ウインドウです。キャラクター作成以外にもマップチップやアイテム、弾の画像などを作るときにも使います。
-  マップ作成ウインドウを開きます。キャラクター作成ウインドウで作成した 8x8、または 16x16 の画像を使い、マップ画面を作成することができます。

第4章 ゲーム画面に自分で描いたキャラを登場させる

-  音源作成ツールです。扱いが少し難しいので今回はスキップします。
-  音楽作成ツールです。音源作成ツールで作成した音源を組み合わせで再生させるためのツールです音源作成ツール同様扱いが少し難しいので今回はスキップします。

4.3 ドット絵を表示してみよう

それでは、保存したドット絵を表示してみましょう。以下のプログラムを「sample01.py」という名前で作成して記述してください。

リスト 4.1: sample01.py

```
1: import pyxel
2: WIDTH = 256
3: HEIGHT = 256
4: IMG_NO = 0
5: pyxel.init(WIDTH, HEIGHT)
6: pyxel.load('mychara.pyxres')
7: pyxel.cls(1)
8: pyxel.blit(WIDTH / 2, HEIGHT / 2, IMG_NO, 0, 0, 16, 16)
9: pyxel.show()
```

プログラムが記述できたら、F5 キーまたは Visual Studio Code 上部メニューの「Debug」→「Start Debugging」でデバッグ実行します。

第4章 ゲーム画面に自分で描いたキャラを登場させる

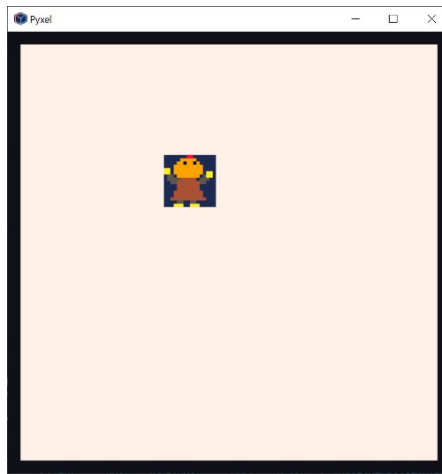


図 4.6: キャラを描く①

見事アンパンマンが表示されました。今回新しく登場したコマンド命令を見ていきましょう。

```
6: pyxel.load('mychara.pyxres')
```

先ほど作成したリソースファイル「mychara.pyxres」を読み込みます。この命令以降、作成したイメージデータを使うことができるようになります。

```
8: pyxel.blt(WIDTH / 2, HEIGHT / 2, IMG_NO, 0, 0, 16, 16)
```

ゲーム画面に画像を配置するための命令です。命令の意味は次のようになります。

```
pyxel.blt(  
  キャラの左上の X 座標,
```

第4章 ゲーム画面に自分で描いたキャラを登場させる

キャラの左上の y 座標,
イメージ番号,
指定したイメージ番号の切り出し x 座標開始位置,
指定したイメージ番号の切り出し y 座標開始位置,
切り出すイメージの幅,
切り出すイメージ高さ,
透明にする色※指定する場合のみに指定します)

「透明にする色」とはなんのでしょうか？ ゲーム画面に表示したキャラクターを見てみましょう。現在、アンパンマンの周りの色（1:ネイビー）がそのまま表示されており、ちょっと格好悪いです。この周りの色を「透明」にすることができます。blt 命令を以下のように書き換えてみましょう。

```
pyxel.blit(WIDTH / 2, HEIGHT / 2, IMG_NO, 0, 0, 16, 16, 1)
```

透明色に「1：ネイビー」を指定すると、

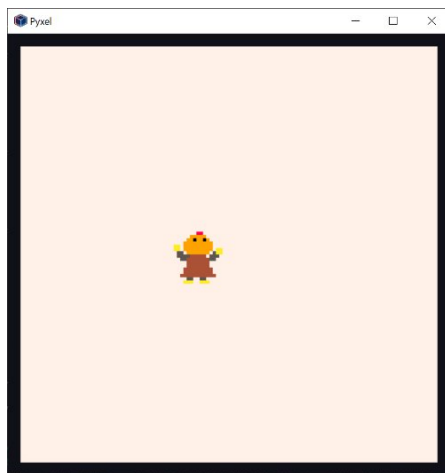


図 4.7: キャラを描く②

このようにアンパンマンの周りの背景が透過して表示することができます。

4.4 ドット絵を動かしてみよう

次に表示したキャラクターを動かしてみたいと思います。今までは、「show」という関数で文字や画像を表示していただけですが、実際にゲームを作るときは、次のプログラムのように「クラス化」という方法を使い、「draw」関数と「update」関数に動作させたい内容を記述します。（「draw」関数と「update」関数を繰り返し実行させることで、画面の更新やゲームの状態を更新させていきます）update 関数と draw 関数の使い分けは、

- update 関数
 - － 説明：自分や敵のキャラクターの位置の移動や、あたり判定（※後述）やキーボード入力（※後述）などを行います。多少重い処理を記述しても OK です。この関数は通常 1 秒間に 30 回実行されます。
 - － 記述する内容：
 - * キャラクターや弾の座標の計算処理
 - * マウスやキーボードによる入力処理
 - * あたり判定
- draw 関数
 - － 説明：画面に描く文字や絵などの「描画」に関する処理を記述します。この関数内では重い処理は記述してはいけません。この関数は通常 1 秒間に 30 回実行されます。
 - － 記述する内容：
 - * 画面クリア (cls)
 - * マップ画像の表示
 - * キャラクター画像、弾画像の表示

それでは、update 関数と draw 関数を使って「App」という名前のクラスを作成し、ゲーム作成の基本部分（骨組み）を作ってみましょう。新規ファイルで「sample02.py」を作成し、以下のプログラムを記述してください。

リスト 4.4: sample02.py

第4章 ゲーム画面に自分で描いたキャラを登場させる

```
1: import pyxel
2:
3: WIDTH = 128
4: HEIGHT = 128
5: IMG_NO = 0
6:
7: class App:
8:     my_x = 0
9:     my_y = 0
10:
11:     def __init__(self):
12:         pyxel.init(WIDTH, HEIGHT)
13:         pyxel.load('mychara.pyxres')
14:         pyxel.run(self.update, self.draw)
15:
16:     def update(self):
17:         self.my_x = pyxel.mouse_x
18:         self.my_y = pyxel.mouse_y
19:
20:     def draw(self):
21:         pyxel.cls(7)
22:         pyxel.blit(self.my_x, self.my_y, IMG_NO, 0, 0, 16, 16, 1)
23:
24: App()
```

このプログラムを実行すると、ゲーム画面上にマウスを移動するとマウスの位置にアンパンマンが表示され、マウス追従してアンパンマンが移動します。

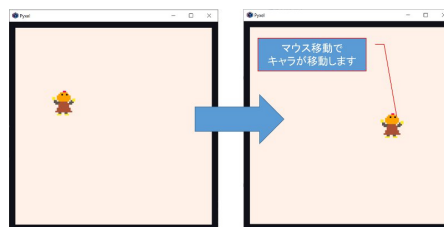


図 4.8: キャラが動く

それでは、import より下のプログラムを説明していきます。

第4章 ゲーム画面に自分で描いたキャラを登場させる

```
3: WIDTH = 128
4: HEIGHT = 128
5: IMG_NO = 0
```

ここでは画面サイズ (幅 128 ドット、縦 128 ドット) イメージ番号 (0) を指定します。

```
7: class App:
8:     my_x = 0
9:     my_y = 0
10:     .....
```

「class App:」で App という名前のクラスを定義します。「my_x = 0」と「my_y = 0」は変数定義です。この App クラスに

- my_x
- my_y

を定義します。この変数は後で出てくる「update」関数と「draw」関数は「self.my_x」と「self.my_y」という変数名でアクセスすることができます。

```
11:     def __init__(self):
12:         pyxel.init(WIDTH, HEIGHT)
13:         pyxel.load('mychara.pyxres')
14:         pyxel.run(self.update, self.draw)
```

「def __init__(self):」で__init__という特殊な初期化のための関数を作成します。この__init__関数はクラスを新規で作成するときに自動的に実行される関数です。今回のプログラムでは

第4章 ゲーム画面に自分で描いたキャラを登場させる

```
class App:
    .....

App()
```

という記述をしていて、「class App:」で定義した App クラスを「App()」の箇所ですべて App クラスを新規作成します。このとき、「__init__」関数が自動的に実行される、という仕組みになります。

ちょっと分かりづらいですね。今は「このような書き方すれば、App クラスを新規作成して、__init__関数が自動的に実行されるんだな」くらいの認識でOKです。

それでは、いよいよ__init__関数の中身をみていきましょう。

```
12: pyxel.init(WIDTH, HEIGHT)
```

pyxel.init 関数でゲーム画面の縦横のドットサイズを決めます。

```
13: pyxel.load('mychara.pyxres')
```

pyxel.load 関数で同一フォルダに存在するリソースファイル「mychara.pyxres」を読み込みます。この命令以降、作成したイメージデータを扱うことができます。

```
14: pyxel.run(self.update, self.draw)
```

pyxel.run 関数で「App」クラス内に作成（＝定義）する「update」関数と「draw」関数を pyxel に組み込む、という命令です。この命令以降、指定した

第4章 ゲーム画面に自分で描いたキャラを登場させる

「update」関数と「draw」関数を約 0.03 秒に 1 回づつ (=1 秒間に 30 回) 実行し続けます。

それでは 1 秒間に 30 回実行される「update」関数と「draw」関数を見ていきましょう。まずは「update」関数から。

```
def update(self):
    self.my_x = pyxel.mouse_x
    self.my_y = pyxel.mouse_y
```

「pyxel.mouse_x」と「pyxel.mouse_y」はそれぞれゲーム画面上のマウスの X 位置、Y 位置の値を示しております。この 2 つの値をそれぞれクラス変数の「my_x」と「my_y」に格納することで、draw 関数側で、マウス座標位置で更新された「my_x」と「my_y」の値を読み込むことができます。

最後に「draw」関数をみていきます。

```
def draw(self):
    pyxel.cls(7)
    pyxel.blit(self.my_x, self.my_y, IMG_NO, 0, 0, 16, 16, 1)
```

まず、「pyxel.cls(7)」の命令で 7 = 白色で画面全体をクリアします。次に「pyxel.blit(self.my_x, self.my_y, IMG_NO, 0, 0, 16, 16, 1)」で

- 座標位置 (my_x, my_y) を指定 (※これはマウスの座標位置)
- 「IMG_NO(=0)」で画面に表示するイメージ番号を指定
- イメージ番号の座標 (0, 0) から高さ 16 ドット、幅 16 ドットの画像を使う
- 描写する画像は色番号「1 (ネイビーブルー)」を透明色とする

という命令を発行します。これで、常にマウスの位置に追従するアンパンマンができました！！

次の章では、いよいよ「敵」を登場させてやっつけてみましょう。

第 5 章

敵キャラ（ばいきんまん）をアンパンチする

ようやく自キャラ（アンパンマン）が動くようになりました。せっかくアンパンマンが動かせるようになったので、敵キャラとして「ばいきんまん」も登場させてアンパンチしてみましょう。

5.1 簡易もぐらたたきゲームを作る

一定間隔（1 秒おき）でばいきんまんが出てきて、それを倒すゲームを作ってみたいと思います。いわゆる「もぐらたたきゲーム」です。とりあえず、簡単化のために一度でてきたもぐら（ばいきんまん）は倒されるまで消さない仕様とします。ゲーム作成までの手順として

- pyxeleditor で敵キャラを描く
- 敵キャラを一定時間で表示するプログラム
- 敵キャラと自キャラの当たり判定（敵を倒す）を行います。

5.2 敵キャラを描く

それでは、また「pyxeleditor」を起動して今度はイメージ番号「1」に敵のキャラクターを描いてみましょう。Visual Studio Code の上部メニューの「Terminal」→「New Terminal」をクリックしてターミナルを開き、「pyxeleditor mychara」

第5章 敵キャラ（ばいきんまん）をアンパンチする

コマンドを実行し、「pyxeleditor」を起動します。

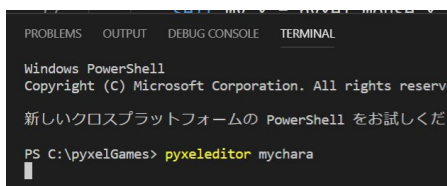


図 5.1: pyxeleditor 起動コマンド

pyxeleditor が起動したらイメージ番号を「1」に切り替えます。画面右下部の「」の「+」をクリックすると、番号が0→1に変わります。この「-」と「+」で編集するイメージ画像の番号を0～2で切り替えることができます。

では、さっそくイメージ番号「1」に敵キャラを描いてみましょう。

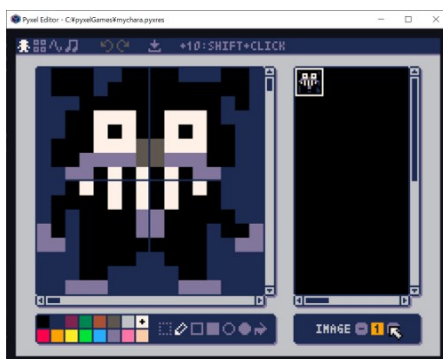


図 5.2: 敵キャラを描く

はい、こんな感じでアンパンマンの敵「ばいきんまん」が描けました。最後に  ボタンを押して保存してください。

5.3 敵キャラを一定周期で表示させる①（準備編）

ゲーム画面に一定周期で敵キャラを表示させるプログラムを「タイマ」で書いてみましょう。今後、タイマは「敵の出現」や「ゲーム時間のカウント」「一定周期での状態監視」など、色々な場面で利用価値があるので、ぜひタイマを使う方法を覚えておきましょう。新規ファイルで「sample05-01.py」を作成して、次のプログラムを記述してください。

リスト 5.1: sample05-01.py

```
1: import pyxel
2:
3: WIDTH = 128
4: HEIGHT = 128
5: IMG_NO = 0
6:
7: class App:
8:     my_x = 0
9:     my_y = 0
10:    interval = 30
11:    cnt = 0
12:
13:    def __init__(self):
14:        pyxel.init(WIDTH, HEIGHT)
15:        pyxel.load('mychara.pyxres')
16:        pyxel.run(self.update, self.draw)
17:
18:    def makeEnemy(self):
19:        print('Create Enemy')
20:
21:    def checkTimer(self):
22:        bRet = False
23:        self.cnt = (self.cnt + 1) % self.interval
24:        if(self.cnt == 0):
25:            bRet = True
26:        return bRet
27:
28:    def update(self):
29:        self.my_x = pyxel.mouse_x
30:        self.my_y = pyxel.mouse_y
31:        if(self.checkTimer() == True):
32:            self.makeEnemy()
33:
```

第5章 敵キャラ（ばいきんまん）をアンパンチする

```
34:     def draw(self):
35:         pyxel.cls(7)
36:         pyxel.blit(self.my_x, self.my_y, IMG_NO, 0, 0, 16, 16, 1)
37:
38: App()
```

それでは新しく追加した処理をみてみましょう。

```
10:     interval = 30
11:     cnt = 0
```

新しく「interval」と「cnt」という変数を定義しております。この2つの変数はタイマの「インターバル間隔値」と「カウンタ」に使われます。この変数は「App」クラス内で「self.interval」「self.cnt」という変数名で使うことができます。

```
18:     def makeEnemy(self):
19:         print('Create Enemy')
```

新しく「makeEnemy」という関数を作成しました。この「makeEnemy」関数を update 関数内で一定周期ごとに実行させるようにします。（※詳細は後述）この段階では、この関数は"Create Enemy"という文字を表示するだけの機能しかありません。タイマの実装が確認できたら、この「makeEnemy」関数に実際に敵キャラを生成する処理を追記していきます。

```
21:     def checkTimer(self):
22:         bRet = False
23:         self.cnt = (self.cnt + 1) % self.interval
24:         if(self.cnt == 0):
25:             bRet = True
26:         return bRet
```

第5章 敵キャラ（ばいきんまん）をアンパンチする

新しく「checkTimer」という関数を作成しました。この「checkTimer」関数は「update」関数内で実行することで一定周期ごとに True を返す関数となります。（一定周期以外のときは False を返す）

一定周期を作るための処理として「self.cnt = (self.cnt + 1) % self.interval」という処理（計算式）が特徴的です。これは約 0.03 秒ごとに「checkTimer」関数が実行され、「(self.cnt + 1)」でカウントアップ(+1)をしつつ、「% self.interval」という余剰計算（＝余り演算）で「self.interval」よりも大きな数字になると、「0」にリセットする計算が実行されます。具体的には「interval」が「3」だとすると「cnt」は「checkTimer」が実行されるたびに「0 → 1 → 2 → 0 → 1 → 2 → 0…」を繰り返すようになります。「cnt」が「0」になったときのみ処理をする命令「if(self.cnt == 0)」を記述することで周期タイマ化することができます。

専門用語でいうとこのようなタイマを「プリスケアラ（分周期）」といいます。もともと存在する 1 秒間に 30 回実行されるタイマ（update）を利用して、他の間隔のタイマを作ることを「プリスケアラ」と呼びます。

今回は「self.interval = 30」としているので、この「checkTimer」を「update」関数の中で使うと「1 秒間に 1 回、True が返ってくるプリスケアラタイマ」が出来上がります。

次は実際に「update」関数内に「checkTimer」を組み込む処理です。

```
31:         if(self.checkTimer() == True):
32:             self.makeEnemy()
```

このように「checkTimer」関数を「update」関数内で使うことで、1 秒間ごとに「self.makeEnemy()」関数を実行する、という処理が完成します。

実際に実行してみると、コンソール側に 1 秒間に 1 回"Create Enemy"という文字が表示され続けます。

5.4 敵キャラを一定周期で表示させる②（実装編）

一定時間ごと（1秒に1回）に特定の処理「makeEnemy」を実行させることに成功しました。次は、いよいよ敵キャラ「ばいきんまん」を登場させていきます。ばいきんまんを登場させる場所（位置）は乱数を使い、ランダムで表示されるようにしてみます。今回、敵キャラをいっぱい生成するので、敵クラス「Enemy」クラスを作り、それぞれの敵の位置などはクラス内で管理するようにしていきます。

とりあえず、今回も新規ファイル「sample05-02.py」を作成して以下のプログラムを記述してください。

リスト 5.6: sample05-02.py

```
1: import pyxel
2: import random
3:
4: WIDTH = 128
5: HEIGHT = 128
6: IMG_NO = 0
7: ENEMY_NO = 1
8:
9: class Enemy:
10:     def __init__(self):
11:         self.x = random.randint(0, WIDTH)
12:         self.y = random.randint(0, HEIGHT)
13:
14: class App:
15:     my_x = 0
16:     my_y = 0
17:     interval = 30
18:     cnt = 0
19:     enemyList = []
20:
21:     def __init__(self):
22:         pyxel.init(WIDTH, HEIGHT)
23:         pyxel.load('mychara.pyxres')
24:         pyxel.run(self.update, self.draw)
25:
26:     def makeEnemy(self):
27:         self.enemyList.append(Enemy())
28:
29:     def checkTimer(self):
```

第5章 敵キャラ（ばいきんまん）をアンパンチする

```
30:         bRet = False
31:         self.cnt = (self.cnt + 1) % self.interval
32:         if(self.cnt == 0):
33:             bRet = True
34:         return bRet
35:
36:     def update(self):
37:         self.my_x = pyxel.mouse_x
38:         self.my_y = pyxel.mouse_y
39:         if(self.checkTimer() == True):
40:             self.makeEnemy()
41:
42:     def draw(self):
43:         pyxel.cls(7)
44:         for i in reversed(range(0, len(self.enemyList))):
45:             x = self.enemyList[i].x
46:             y = self.enemyList[i].y
47:             pyxel.blt(x, y, 1, 0, 0, 16, 16, 1)
48:
49:         pyxel.blt(self.my_x, self.my_y, IMG_NO, 0, 0, 16, 16, 1)
50:
51: App()
```

それでは、新しく追加した箇所を解説していきます。

```
2: import random
```

今回、敵の登場位置をランダムで変えるために乱数用のモジュール「random」を使います。import 命令で「random」モジュールをロードしておきます。

```
6: ENEMY_NO = 1
```

敵のイメージ番号を指定しておきます。ばいきんまんはイメージ番号「1」に保存したので「ENEMY_NO = 1」とすることで、今後プログラム上では「ENEMY_NO」でばいきんまんの画像を扱うことができます。

第5章 敵キャラ（ばいきんまん）をアンパンチする

```
12: class Enemy:
13:     def __init__(self):
14:         self.x = random.randint(0, WIDTH)
15:         self.y = random.randint(0, HEIGHT)
```

今回新たに追加した敵クラス「Enemy」クラスです。現時点では Enemy クラスの関数は「__init__」関数しかありません。（※後述でアニメーション関数などを追加していきます）「Enemy」クラス生成時にランダムで X 座標、Y 座標を決める処理を__init__に書きます。

- self.x = random.randint(0, WIDTH)
- self.y = random.randint(0, HEIGHT)

「self.x」は 0（＝画面の左端の座標）～WIDTH（＝画面の右端）の座標

「self.y」は 0（＝画面の一番上）～HEIGHT（＝画面の一番下）の座標

となり、ゲーム画面内のどこかにばいきんまんを配置するための計算が完了します。

```
12:     enemyList = []
```

生成された敵クラス「Enemy」を保持するための List 変数「enemyList」を定義します。

■List変数とは？

List変数の名前を「enemyList」とすると・・・

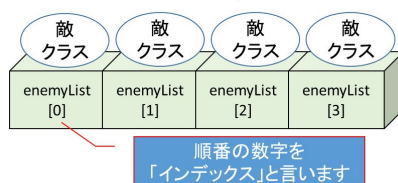


図 5.3: List 変数イメージ①

第5章 敵キャラ（ばいきんまん）をアンパンチする

詳細は後述しますが、List 変数は「append」という関数で新規作成したクラスを追加していったり、delete 関数と一緒に使うことで、簡単に List 内のクラス（※削除する場所「インデックス」を指定）を削除することができる非常に便利なものです。

今回、生成した敵キャラはこの「enemyList」という List 変数で管理していきます。

```
26:     def makeEnemy(self):
27:         self.enemyList.append(Enemy())
```

すでに作成した「makeEnemy」関数の処理を変更します。

「self.enemyList.append(Enemy())」

の命令で「enemyList」変数に新規作成した「Enemy」クラスを追加していきます。append による追加のイメージは次図のような感じです。

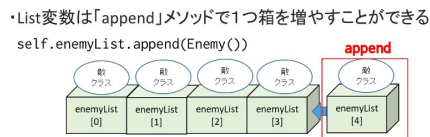


図 5.4: List 変数の追加イメージ

append 実行時に「Enemy」クラスは新規作成され、この新規作成時にランダムで敵キャラの X と Y の座標を計算します。（あとで draw 関数内で使います）

次に draw 関数内の「enemyList」変数の敵キャラ描画に関する部分を見ていきます。

```
44:         for i in range(0, len(self.enemyList)):
45:             x = self.enemyList[i].x
46:             y = self.enemyList[i].y
47:             pyxel.blit(x, y, 1, 0, 0, 16, 16, 1 )
```


第5章 敵キャラ（ばいきんまん）をアンパンチする

「for」というループ（繰り返し）処理と範囲を生成する「range」関数を組み合わせることで、生成済みの敵キャラの個数分だけ敵キャラを描写する、という処理を実現しています。1つずつ解説していきます。

- for i in range(0, len(self.enemyList)):

「range(A, B)」関数というのは「A～B 未満の連続した整数配列を作成する」という関数です。なので、「0～10」の整数配列を作りたい場合は、

```
>>> range(0,11)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

とすることで、整数配列を作ることができます。今回やりたいことは「enemyList」で敵クラス「Enemy」が存在する配列の場所「インデックス値」を取得することです。リスト変数のインデックス値のイメージは図 5.3 のようになります。

もし、図 5.3 のように enemyList のインデックスが [0]～[3] の場合、[0, 1, 2, 3] という配列変数を range で生成することになります。すると、プログラ的には、

```
for i in [0, 1, 2, 3]
```

という意味になり、変数「i」に配列の値を順番に格納しながら for 内の処理を計 4 回繰り返し実行されます。for 内の処理を実行するとき、

- x = self.enemyList[i].x
- y = self.enemyList[i].y

という処理で「enemyList」にある敵クラス「Enemy」から順番に X 座標、Y 座標値を取り出します。（「i」は 0、1、2、3 と順番に実行されます）

最後に「pyxel.blit」の命令でばいきんまんの画像を表示します。を取り出した X 座標、Y 座標値に表示させます。

- pyxel.blit(x, y, 1, 0, 0, 16, 16, 1)

for ループの処理内で取り出した X 座標、Y 座標値に表示させます。

第5章 敵キャラ（ばいきんまん）をアンパンチする

実際にこのプログラムを実行してみましょう。



図 5.5: ゲーム画面

どうでしょうか？ 1秒おきにばいきんまんがゲーム画面に生まれ続ければ成功です！

5.5 いよいよばいきんまんをやっつけする

ようやく、ばいきんまんをやっつける処理「あたり判定」を実装できる段階になりました。今回は、単純に「アンパンマン（自キャラ）がばいきんまん（敵キャラ）の上に乗ったら退治成功」というものにしてみます。まさにもぐらたたき、そのものですね。（アンパンマン＝ハンマー、ばいきんまん＝もぐら）

リスト 5.13: sample05-03.py

```
1: import pyxel
2: import random
3:
4: WIDTH = 128
5: HEIGHT = 128
6: IMG_NO = 0
7:
8: class Enemy:
```

第5章 敵キャラ（ばいきんまん）をアンパンチする

```
9:     def __init__(self):
10:         self.x = random.randint(0, WIDTH)
11:         self.y = random.randint(0, HEIGHT)
12:
13: class App:
14:     my_x = 0
15:     my_y = 0
16:     interval = 30
17:     cnt = 0
18:     enemyList = []
19:
20:     def __init__(self):
21:         pyxel.init(WIDTH, HEIGHT)
22:         pyxel.load('mychara.pyxres')
23:         pyxel.run(self.update, self.draw)
24:
25:     def makeEnemy(self):
26:         self.enemyList.append(Enemy())
27:
28:     def checkTimer(self):
29:         bRet = False
30:         self.cnt = (self.cnt + 1) % self.interval
31:         if(self.cnt == 0):
32:             bRet = True
33:         return bRet
34:
35:     def isCollision(self, _i, _x, _y):
36:         bRet = False
37:         my_x = _x + 8
38:         my_y = _y + 8
39:         if(
40:             (self.enemyList[_i].x <= my_x)
41:             and (self.enemyList[_i].x + 16 >= my_x)
42:             and (self.enemyList[_i].y <= my_y)
43:             and (self.enemyList[_i].y + 16 >= my_y)
44:         ):
45:             bRet = True
46:         return bRet
47:
48:     def update(self):
49:         self.my_x = pyxel.mouse_x
50:         self.my_y = pyxel.mouse_y
51:         if(self.checkTimer() == True):
52:             self.makeEnemy()
53:
54:         for i in reversed(range(0, len(self.enemyList))):
55:             if(self.isCollision(i, self.my_x, self.my_y) == True):
```

第5章 敵キャラ（ばいきんまん）をアンパンチする

```
56:             del self.enemyList[i]
57:
58:     def draw(self):
59:         pyxel.cls(7)
60:         for i in reversed(range(0, len(self.enemyList))):
61:             x = self.enemyList[i].x
62:             y = self.enemyList[i].y
63:             pyxel.blt(x, y, 1, 0, 0, 16, 16, 1 )
64:
65:         pyxel.blt(self.my_x, self.my_y, IMG_NO, 0, 0, 16, 16, 1)
66:
67: App()
```

それでは、今回新規で追加した処理をピックアップしてみましょう。

```
36:     def isCollision(self, _i, _x, _y):
37:         bRet = False
38:         my_x = _x + 8
39:         my_y = _y + 8
40:         if(
41:             (self.enemyList[_i].x <= my_x)
42:             and (self.enemyList[_i].x + 16 >= my_x)
43:             and (self.enemyList[_i].y <= my_y)
44:             and (self.enemyList[_i].y + 16 >= my_y)
45:         ):
46:             bRet = True
```

今回追加した当たり判定用関数「isCollision」を解説します。この関数を説明する前に簡単に当たり判定について説明いたします。

第5章 敵キャラ（ばいきんまん）をアンパンチする

■当たり判定とは？

敵を倒す条件（その逆もあり）のこと

今回は簡単に実装するために「自分キャラのセンター座標が敵の画像の中に入ったら「当たり」とする。



今回は単純に

- 自分のキャラは「センター座標」＝アンパンマン画像の真ん中
- 敵キャラは「16x16」の敵画像のどこでも当たり判定範囲

として、簡単化したいと思います。（もっとちゃんとした当たり判定を実装したい場合は google で「円 当たり判定」と検索すると、よりユーザのイメージに近い正確な当たり判定の計算式がわかります）

この関数での処理は「アンパンマンの中心位置（センター座標）」が「ばいきんまんの画像の中」に入ったら、True、それ以外の場合は False を返す、という処理を行っています。

この当たり判定関数「isCollision」を 0.03 秒ごとに実行される「update」関数内でチェックします。それでは update 関数の中を見ていきましょう。

```
54:     for i in reversed(range(0, len(self.enemyList))):
55:         if(self.isCollision(i, self.my_x, self.my_y) == True):
56:             del self.enemyList[i]
```

またまた for によるループ処理ができました。ただし、今回は「reversed」という関数がついています。この「reversed」関数は「配列の順番を逆にする」という効果があります。実際にどんな処理になるか見てみましょう。

第5章 敵キャラ（ばいきんまん）をアンパンチする

```
>>> list(reversed(range(0, 11)))  
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

この例では画面表示しやすいように「reversed」を list 関数で画面表示可能な形式に変換をしています。

なぜ「順番を逆にする」必要があるのか？ を説明します。今回「update」関数内で enemyList に対し、当たり判定を行います。もし、当たり判定で「当たり = True」となった場合、enemyList から del 関数を使って対象の「Enemy」クラスを削除してしまうとします。

List変数は「del」という外部メソッドで
指定した場所(インデックス)の箱を削除できる

```
del self.enemyList[2]
```

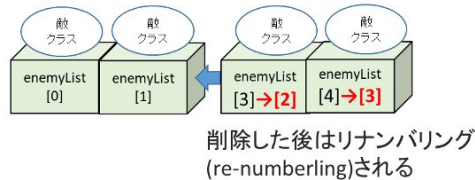


図 5.7: List 変数の削除イメージ

このイメージ図のように List 変数の中間値のインデックス値のクラスを削除してしまった場合、インデックス値がリナンバリングされるため、0 から順番に昇順（＝小さい順）で当たり判定チェック&削除処理をしてしまうと、意図しない（インデックス値がずれてしまう）動きになってしまいます。

そこで、逆順に当たり判定&削除処理を行うことで、このインデックスずれを解消する効果があります。

- `if(self.isCollision(i, self.my_x, self.my_y) == True):`

ループ処理内で順番に（逆順で）enemyList 内の敵クラス「Enemy」の当たり判定を行っていきます。当たり判定した結果、True = 当たり、となった Enemy クラスを以下の処理で削除します。

第5章 敵キャラ（ばいきんまん）をアンパンチする

- `del self.enemyList[i]`

「del」関数は指定した List 変数の要素を削除することができます。
それでは、実際にこのプログラムを実行してみましょう。

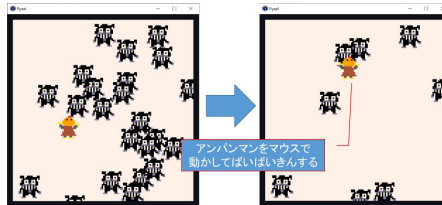


図 5.8: 敵を倒すアクション実装

アンパンマンをばいきまんの位置に移動すると、見事ばいきまんを退治することができました！！

第 6 章

アニメーションをやってみよう

ここまでで、自キャラ（アンパンマン）で敵キャラ（ばいきんまん）をやっつける、というゲームのベースを作ることができました。「あれ？ アンパンチしてなくね？」と突っ込まれるとその通りなので、この章では、実際にキャラクターにアニメーションを実装する方法を紹介します。

アニメーションを作成するには基本的には「パラパラ漫画方式」で実現します。つまりは、絵の一部を変えた複数のキャラクター画像を用意しておき、それをタイマで順番に表示されるように制御する、という方法です。では実際にアンパンマンの必殺技「アンパンチ」のアニメーションを作りながら学習してみたいと思います。（ついでにばいきんまんがやられるアニメーションも描いてみましょう）

6.1 アンパンチのアニメーションの作り方（手順）

今回アニメーションを作成する手順は以下のような感じです

- pyxeleditor でアンパンチするアンパンマンの絵を 6 枚描く
- アニメーションテスト用のプログラムでアニメーション動作をチェック
- 実際のゲームにアニメーションを取り込んでみる

となります。

6.2 pyxeleditor でアンパンチ画像を作成する

それでは、Visual Studio Code の上部メニューの「Terminal」→「New Terminal」をクリックし、画面内にターミナルを表示します。ターミナルウィンドウで

```
pyxeleditor mychara
```

と実行し、pyxeleditor を再度起動します。今回は、イメージ番号「0」の自キャラ（アンパンマン）に新たに 16×16 の画像を 5 枚追加します。

アニメーション画像を作成する際に便利な編集ツールは「

pyxeleditor の右側のウィンドウに注目してください。スクロールバーが X 方向と Y 方向にありますね。実は、pyxeleditor のイメージ画像は 16×16 の画像を同じイメージ番号にいっぱい描くことができます。1つのイメージ番号で描くことができる画像のサイズは「 255×255 」ドットとなっているようようです。

ゲーム画面にキャラを表示する「`pyxel.blit`」をおさらいします。

```
pyxel.blit(  
    キャラの左上の x 座標,  
    キャラの左上の y 座標,  
    イメージ番号,  
    指定したイメージ番号の切り出し x 座標開始位置,  
    指定したイメージ番号の切り出し y 座標開始位置,  
    切り出すイメージの幅,  
    切り出すイメージ高さ,  
    透明にする色※指定する場合のみに指定します)
```

この関数で指定している 4、5 番目のパラメータ、

第6章 アニメーションをやってみよう

- 指定したイメージ番号の切り出し X 座標開始位置
- 指定したイメージ番号の切り出し Y 座標開始位置

を変えることで、どの場所の画像を取り出すかを変更することが可能なのです。

では、さっそく選択ツールを使って、1枚目のアンパンマンの右横にコピーアンパンマンを描いてみたいと思います。選択ツールの使い方は、コピーしたい画像のドット位置からマウスでドラッグ&ドロップして範囲を選択します。



図 6.1: pyxeleditor で選択ツールを使う①

こんな感じでコピーしたい範囲を選択します。選択した状態でキーボードのコピーコマンド「Ctrl キー + C キー」を押します。これでクリップボードに選択した画像（今回の場合は 16×16 の1枚目のアンパンマン画像がコピーされます。

次にクリップボードにコピーした画像を貼り付けます。pyxeleditor の左側の画像の「1枚目」の横の位置をクリックします。

第6章 アニメーションをやってみよう



図 6.2: pyxeleditor で選択ツールを使う②

左側にはまだ何も絵がありません。この状態でキーボードの貼り付けコマンド「Ctrl キー+ V キー」を押します。貼り付けに成功すると、次図のように 1 枚目のアンパンマンが右横にもコピーされます。

ここまでくれば、あとはパラパラ漫画を作るだけです。コピーした画像の一部（今回は右手の画像部分）を少し修正します。



図 6.3: pyxeleditor でパラパラ漫画を作る①

ちょっと手の位置を変えただけの画像を作成することができました。この作業をあと、何度か繰り返して計 6 枚のアンパンマン画像を横ならびに用意していきます。イメージ的には次図のような感じです。

第6章 アニメーションをやってみよう



図 6.4: pyxeleditor でパラパラ漫画を作る②



ここまでできましたら「」ボタンのクリックをして画像を保存します。
(※よく忘れるので注意)

6.3 アニメーションをテストしてみる

それでは、pyxeleditor で作成したパラパラ漫画がうまくいくかテスト用プログラムでテストをしてみたいと思います。Visual Studio Code で新規ファイル「animation01.py」を作成し、以下のコードを入力してください。

リスト 6.1: animation01.py

```
1: import pyxel
2:
3: WIDTH = 128
4: HEIGHT = 128
5: IMG_NO = 0
6:
7: pyxel.init(WIDTH, HEIGHT, fps=10)
8: pyxel.load('mychara.pyxres')
9:
10: ani_no = 0
11: ani_max = 6
12:
13: while True:
14:     pyxel.cls(1)
15:     ani_no = ( ani_no + 1 ) % ani_max
16:     pyxel.blt(
17:         WIDTH / 2, HEIGHT / 2,
18:         IMG_NO,
19:         0 + (ani_no * 16), 0,
20:         16, 16)
21:     pyxel.flip()
```

第6章 アニメーションをやってみよう

今回のアニメーションのテストプログラムは通常のゲームの作りよりも単純化しております。ゲーム本体クラスを作らずに描画テストを行う場合、「`pyxel.init`」で `fps` 値の調整し「`while`」と「`pyxel.flip`」関数の組み合わせで実現ができてしまいます。少しプログラムの説明をいたします。

```
7: pyxel.init(WIDTH, HEIGHT, fps=10)
```

今回、「`pyxel.init`」で初めて「`fps`」の値を変更しております。`fps` は指定しない場合「30」が使われます。この `fps` とは「フレーム/秒」の意味で「1秒間に何フレーム（＝`draw` 関数、`update` 関数）を実行するか」を定義することができます。

アニメーションを実装する際にはタイマ動作でプリスケアラを使い、アニメーションを実現しますが、今回のようなアニメーションをテストするだけのプログラムの場合は、`fps` 値を変更すればわざわざタイマを作らなくて済むのでテストが楽になります。

今回指定した `fps` は「10」なので、通常の「30」よりも $1/3$ の速度で画面が描写されるようになります。

```
10: ani_no = 0
11: ani_max = 6
```

今回新しくでてきた変数の意味を解説します。

- `ani_no` : 現在表示中のアニメーション画像番号（今回の場合は 0～5）
- `ani_max` : アニメーション画像の最大値 + 1（今回の場合は 6）

アンパンマンのアニメーション画像は合計 6 枚で、アニメーション画像は 0 始まりでカウントすると 0、1、2、3、4、5 の 6 つになります。`ani_max` は単純にアニメーション枚数（6 枚）を入れてもらえば OK です。

第6章 アニメーションをやってみよう

```
13: while True:
```

「while」はfor同様ループ処理です。ただし「while」の場合は終了条件無しで利用することができます。このプログラムのように「while True」とすると、無限ループになります。

```
15:     ani_no = ( ani_no + 1 ) % ani_max
16:     pyxel.blit(
17:         WIDTH / 2, HEIGHT / 2,
18:         IMG_NO,
19:         0 + (ani_no * 16), 0,
20:         16, 16)
```

「ani_no = (ani_no + 1) % ani_max」の計算で1秒間に10回ごとにani_noは「0 → 1 → 2 → 3 → 4 → 5 → 0 → 1 → 2…」を繰り返します。「pyxel.blit」の4番目の引数に注目してください。「0 + (ani_no * 16)」という変数を4番目のパラメータ「指定したイメージ番号の切り出し X 座標開始位置」に指定しています。この計算で、この4番目のパラメータは「0 → 16 → 32 → 48 → 64 → 80 → 0 → 16 → 32…」を繰り返すようになります。これで、イメージ番号「0」の画像取り出し X 座標位置を順番に変えることができます。

```
17:     pyxel.flip()
```

「pyxel.flip」は強制的に画面描写を行う、という命令です。今回の場合は1秒間で10回の画面描写となっているので、「pyxel.flip」は1秒間で10回動作することができます。（＝この関数で次の描写までの時間を待機する）

実際にプログラムを実行してみてください。

第6章 アニメーションをやってみよう

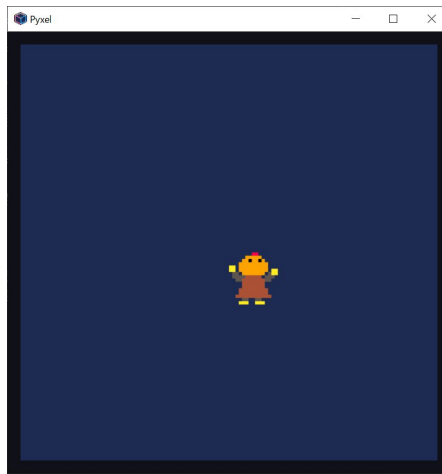


図 6.5: アンパンチアニメ

どうでしょうか？ 何度もアンパンマンがアンパンチするアニメーションが表示されたら成功です！

6.4 ついでにばいきんまんの画像とアニメーションを作ってみる

せっかくなので、やっつけられてしまう敵キャラの「ばいきんまん」のアニメーションを作ってみましょう。手順はアンパンマンのときと同様にまず「pyxeleditor」でイメージ番号「1」に複数のアニメーション用画像を用意します。とりあえず、アンパンチされて、ばいきまんが真っ二つになって消滅するアニメーションを作ってみました（笑）

第6章 アニメーションをやってみよう

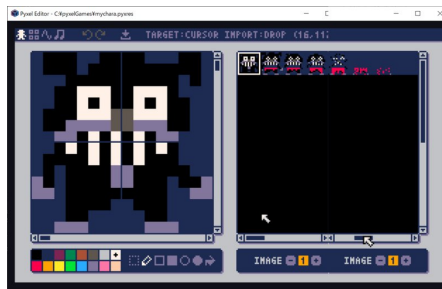


図 6.6: pyxeleditor でパラパラ漫画を作る③

今回は全部で8枚のアニメーション画像になりました。では、先ほどのアニメーションテストプログラムで以下のように一部修正すればばいきまんのアニメーションも確認できます。

```
5: IMG_NO = 0 → IMG_NO = 1
```

```
11: ani_max = 6 → ani_max = 8
```

改造できたら早速実行してみてください。

第6章 アニメーションをやってみよう

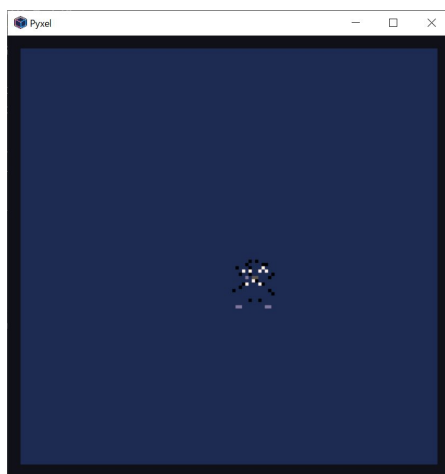


図 6.7: ばいばいきんアニメ

ばいきまんの消滅アニメーションがうまくいけば完成です。いよいよ実際のプログラムに実装する準備に入りたいと思います。

6.5 実際のプログラムにアニメーションを組み込んでみる (キャラクラス)

それでは、アニメーションがちゃんと動作することが確認できたら、実際のプログラムにアンパンチアニメーションを実装してみましょう。

ここからはプログラムが大きくなってしまいますので、

- 自キャラクラス
- 敵キャラクラス

をまとめた「pyxelChara.py」ファイルをまずは作成したいと思います。(後にゲームメインクラス「App」でそれらのキャラクラスをロードさせます) この「自キャラクラス」「敵キャラクラス」にはそれぞれアニメーションを実行するための機能を実装します。

リスト 6.9: pyxelChara.py

第6章 アニメーションをやってみよう

```
1: import pyxel
2: import random
3:
4: IMG_NO = 0
5: ENEMY_IMG_NO = 1
6:
7: class MyChara:
8:     ani_no = 0
9:     ani_max = 6
10:    ani_interval = 2
11:    ani_cnt = 0
12:    ani_flg = False
13:
14:    def __init__(self):
15:        self.x = pyxel.mouse_x
16:        self.y = pyxel.mouse_y
17:        self.ani_flg = False
18:
19:    def startAnimation(self):
20:        self.ani_cnt = -1
21:        self.ani_no = 0
22:        self.ani_flg = True
23:
24:    def drawImage(self, _my_x, _my_y):
25:        if(self.ani_flg == True):
26:            self.ani_cnt = ( self.ani_cnt + 1 ) % self.ani_interval
27:            if(self.ani_cnt == 0):
28:                self.ani_no = self.ani_no + 1
29:                if(self.ani_no >= self.ani_max):
30:                    self.ani_no = 0
31:                    self.ani_flg = False
32:            else:
33:                self.ani_no = 0
34:        pyxel.blt(
35:            _my_x, _my_y,
36:            IMG_NO,
37:            0 + (self.ani_no * 16), 0,
38:            16, 16, 1)
39:
40: class Enemy:
41:     ani_no = 0
42:     ani_max = 8
43:     ani_interval = 2
44:     ani_cnt = 0
45:     ani_flg = False
46:     delete_flg = False
```

第6章 アニメーションをやってみよう

```
47:
48:     def __init__(self, _width, _height):
49:         self.x = random.randint(0, _width)
50:         self.y = random.randint(0, _height)
51:         self.delete_flg = False
52:
53:     def startAnimation(self):
54:         if(self.ani_flg == False):
55:             self.ani_cnt = -1
56:             self.ani_no = 0
57:             self.ani_flg = True
58:
59:     def drawImage(self, _x, _y):
60:         if(self.ani_flg == True):
61:             self.ani_cnt = ( self.ani_cnt + 1 ) % self.ani_interval
62:             if(self.ani_cnt == 0):
63:                 self.ani_no = self.ani_no + 1
64:                 if(self.ani_no >= self.ani_max):
65:                     self.ani_no = self.ani_max - 1
66:                     self.ani_flg = False
67:                     self.delete_flg = True
68:             else:
69:                 self.ani_no = 0
70:             pyxel.blt(
71:                 _x, _y, ENEMY_IMG_NO,
72:                 0 + (self.ani_no * 16), 0,
73:                 16, 16, 1)
```

自キャラクラスの解説

まず、自キャラクラス「MyChara」の解説をしていきます。「MyChara」クラスには以下のクラス変数があります。

```
8:     ani_no = 0
9:     ani_max = 6
10:    ani_interval = 1
11:    ani_cnt = 0
12:    ani_flg = False
```

第6章 アニメーションをやってみよう

それぞれの変数の意味は以下の通りです。

- ani_no :現在表示するキャラ画像のアニメーションのインデックス値 (0 ~5)
- ani_max : アニメーションを最大枚数
- ani_interval :アニメーションのための周期インターバル値 (※「1」を指定すると 0.06 秒ごとに実行される)
- ani_cnt : アニメーションのための周期カウント値
- ani_flg :アニメーションを開始する場合、True にする (アニメーション終了時に False にしておく)

これら変数を使い、自キャラのアニメーションを実現させます。

次に自キャラ「MyChara」クラスの初期化関数です。

```
14:     def __init__(self):
15:         self.x = pyxel.mouse_x
16:         self.y = pyxel.mouse_y
17:         self.ani_flg = False
```

自キャラクラス「MyChara」の初期化処理として自キャラの最初の初期位置をマウスの X 座標、Y 座標から取得し、アニメーション実行中フラグを OFF(=False) にしておきます。

```
19:     def startAnimation(self):
20:         self.ani_cnt = -1
21:         self.ani_no = 0
22:         self.ani_flg = True
```

自キャラクラスに「startAnimation」という関数を追加します。この関数を実行すると、アニメーション状態を一度リセットし、アンパンチを行うための変数をセットしていきます。具体的には「self.ani_flg = True」によって、同じ

第6章 アニメーションをやってみよう

自キャラ関数である「drawImage」関数（※後述）で周期タイマ処理が実行されます。

```
24:     def drawImage(self, _my_x, _my_y):
25:         if(self.ani_flg == True):
26:             self.ani_cnt = ( self.ani_cnt + 1 ) % self.ani_interval
27:             if(self.ani_cnt == 0):
28:                 self.ani_no = self.ani_no + 1
29:                 if(self.ani_no >= self.ani_max):
30:                     self.ani_no = 0
31:                     self.ani_flg = False
32:             else:
33:                 ani_no = 0
34:             pyxel.blt(
35:                 _my_x, _my_y,
36:                 IMG_NO,
37:                 0 + (self.ani_no * 16), 0,
38:                 16, 16, 1)
```

自キャラのアニメーションを実現させるために自キャラクラスに「drawImage」関数を追加します。この関数はアニメーション実行中ではない場合（「ani_flg」が False の場合）は1枚目のアンパンマン画像を表示します。アニメーション実行中（「ani_flg」が True）の場合は、アニメーションの切り替えタイミング（「ani_cnt」の値）を確認してアニメーション画像を表示するようにしています。

アニメーションが終了したら、繰り返しアニメーションが実行されないようにアニメーション実行中フラグ「ani_flg」を False にして、これ以降はアニメーションされないようにしています。

再度アニメーションをするには「startAnimation」の関数が呼ばれたときに再度アニメーションが開始されます。この「startAnimation」はこの後説明する「update」関数内の当たり判定時に実行します。

敵キャラクラスの解説

次に、敵キャラクラス「Enemy」の解説をしていきます。「Enemy」クラスには以下のクラス変数があります。

第6章 アニメーションをやってみよう

```
37:     ani_no = 0
38:     ani_max = 8
39:     ani_interval = 2
40:     ani_cnt = 0
41:     ani_flg = False
42:     delete_flg = False
```

それぞれの変数の意味は以下の通りです。

- ani_no : 現在表示するキャラ画像のアニメーションのインデックス値 (0 ~ 7)
- ani_max : アニメーションを最大枚数 (敵キャラは 8 枚)
- ani_interval : アニメーションのための周期インターバル値 (※「1」を指定すると 0.06 秒ごとに実行される)
- ani_cnt : アニメーションのための周期カウント値
- ani_flg : アニメーションを開始する場合、True にする (アニメーション終了時に False にしておく)
- delete_flg: 「やっつけられた」アニメーション終了後、フラグを True にして、「update」関数での削除処理を待つ

これら変数を使い、敵キャラのアニメーションを実現させます。

6.6 実際のプログラムにアニメーションを組み込んでみる (メインクラス)

それでは、仕上げです。自キャラと敵キャラをゲームメインクラスに組み込んでみましょう。新規ファイル sample06-01.py を作成し、次のプログラムを記述してください。

リスト 6.15: sample06-01.py

```
1: import pyxel
2: import random
3: import pyxelChara as Chara
4:
```

第6章 アニメーションをやってみよう

```
5: WIDTH = 128
6: HEIGHT = 128
7:
8: class App:
9:     my_x = 0
10:    my_y = 0
11:    interval = 30
12:    cnt = 0
13:    enemyList = []
14:
15:    def __init__(self):
16:        pyxel.init(WIDTH, HEIGHT)
17:        pyxel.load('mychara.pyxres')
18:        self.myChara = Chara.MyChara()
19:        pyxel.run(self.update, self.draw)
20:
21:    def makeEnemy(self):
22:        self.enemyList.append(Chara.Enemy(WIDTH, HEIGHT))
23:
24:    def checkTimer(self):
25:        bRet = False
26:        self.cnt = (self.cnt + 1) % self.interval
27:        if(self.cnt == 0):
28:            bRet = True
29:        return bRet
30:
31:    def update(self):
32:        self.my_x = pyxel.mouse_x
33:        self.my_y = pyxel.mouse_y
34:        if(self.checkTimer() == True):
35:            self.makeEnemy()
36:
37:        for i in reversed(range(0, len(self.enemyList))):
38:            if(self.isCollision(i, self.my_x, self.my_y) == True):
39:
40:                self.myChara.startAnimation()
41:                self.enemyList[i].startAnimation()
42:
43:                if(self.enemyList[i].delete_flg == True):
44:                    del self.enemyList[i]
45:
46:    def isCollision(self, _i, _x, _y):
47:        bRet = False
48:        my_x = _x + 8
49:        my_y = _y + 8
50:        if(self.enemyList[_i].ani_flg != True
51:           and self.enemyList[_i].delete_flg == False):
```

第6章 アニメーションをやってみよう

```
52:
53:         if(
54:             (self.enemyList[_i].x <= my_x)
55:             and (self.enemyList[_i].x + 16 >= my_x)
56:             and (self.enemyList[_i].y <= my_y)
57:             and (self.enemyList[_i].y + 16 >= my_y)
58:         ):
59:             bRet = True
60:         return bRet
61:
62:     def draw(self):
63:         pyxel.cls(7)
64:         for i in reversed(range(0, len(self.enemyList))):
65:             x = self.enemyList[i].x
66:             y = self.enemyList[i].y
67:             if(self.enemyList[i].delete_flg == False):
68:                 self.enemyList[i].drawImage(x, y)
69:
70:         self.myChara.drawImage(self.my_x, self.my_y)
71: App()
```

それでは新規追加/変更したプログラムの箇所を解説していきます。

```
3: import pyxelChara as Chara
```

先ほど作成したキャラクラスをまとめた python ファイル「pyxelChara.py」をモジュール名「Chara」として読み込みます。この import 処理以降、

- Chara.MyChara
- Chara.Enemy

という形で、作成したそれぞれのキャラクラスを使うことができます。

```
18: self.myChara = Chara.MyChara()
```


第6章 アニメーションをやってみよう

メインゲームクラスの初期化関数「__init__」内で pyxel ライブラリのロード後に自キャラクラス「MyChara」を「self.myChara」変数名で新規作成します。これ以降、「self.myChara. 関数名」の形で自キャラクラスのアニメーション関数などを実行することができます。

リスト 6.23: makeEnemy 関数内

```
23:         self.enemyList.append(Chara.Enemy(WIDTH, HEIGHT))
```

「makeEnemy」関数内の処理をこのように書き換えます。敵キャラを新規で作成する処理です。意味は以前のものをほとんど変わらないので（ゲーム画面の幅と高さのパラメータが増えただけ）説明は省きます。

次は update 関数内の「当たり判定」を改造します。

リスト 6.23: update 関数内

```
41:         self.myChara.startAnimation()
42:         self.enemyList[i].startAnimation()
43:
44:         if(self.enemyList[i].delete_flg == True):
45:             del self.enemyList[i]
```

前のプログラムでは当たり判定後、すぐに「del」関数で対象の Enemy クラスを削除すれば OK、だったのですが、今回は、

「敵の当たり判定を判断し、当たった場合」

→「自キャラのアニメーション開始、敵の消滅するアニメーションも開始」

→「敵の消滅アニメーション完了後、削除フラグを True にし、Update 関数で削除されるのを待つ」

という順番になるため、すぐに削除をすることができません。

次は isCollision 関数内でも「当たり判定」の際にフラグを考慮します。

リスト 6.23: update 関数内

第6章 アニメーションをやってみよう

```
41:
42: //firstlinenum[23]
43: //listnum[] [] [python]{
44:     if(self.enemyList[_i].ani_flg != True
45:         and self.enemyList[_i].delete_flg == False):
```

「isCollision」関数で考慮すべきは「ani_flg」と「delete_flg」の2つのフラグ変数です。敵キャラクラスにおいて、

- アニメーション実行中 (ani_flg == True)
 - すでに死亡確定の敵キャラなので、当たり判定はしてはいけない
- 削除フラグが True のクラス (delete_flg == True)
 - すでに死亡した（はず）の敵キャラなので、当たり判定はしてはいけない

の考慮を当たり判定時に考慮する必要があるわけです。すでに死亡した敵キャラに何度も当たり判定をしてしまうと、倒した敵の数のカウントや、自キャラのアニメーション実行がうまくいかず、バグってしまいます。

```
67:         if(self.enemyList[i].delete_flg == False):
68:             self.enemyList[i].drawImage(x, y)
```

こちらも当たり判定同様、フラグ状態をチェックして敵キャラを表示制御しなくてはなりません。タイミング的にまだ実際の「del」処理はしていないが、delete_flg が立っているもの＝すでに消滅アニメーションは完了しているので、敵キャラを表示してはいけません。delete_flg が False(まだ死んでいないか、消滅アニメーション中) のもののみ Enemy クラスの「drawImage」関数を使ってゲーム画面に表示させます。

```
70:     self.MyChara.drawImage(self.my_x, self.my_y)
```

第6章 アニメーションをやってみよう

自キャラクラスの表示には「MyChara」の「dramImage」関数を呼ぶことで、アニメーション実行前か実行中を関数の内部で判断し、画像を表示することができます。

実際にプログラムを実行してみてください。



図 6.8: アニメ付きゲーム画面

アンパンマンがばいきんまんに重なったときアンパンチをかまして、ばいきんまんが消滅する動きが確認できましたでしょうか？

お疲れ様でした。これで、ゲームのメインの部分はほぼ完了しました。あとは、仕上げとして

- 背景（「マップチップ」と「マップデータ」）の作成

を行えばゲームの完成です！ あともう少しお付き合いください。

第7章

背景を作成する

ようやく、ゲームのベース部分はほぼ完成しました。後はゲームの完成度をあげるために背景を用意しましょう。

もし過去に自作ゲームとかで「RPG ツクール」のようなゲーム作成ツールを使った人はよく知っているかもしれませんが。背景画像の作成には「マップチップ」と呼ばれるタイル状の画像を用意し、それを丁寧に配置していくことで背景画面を作ることができます。結構地道な作業になります。気合い入れて頑張りましょう！

背景画面作成のイメージは以下の感じです。

それでは手順を以下に紹介します。

- pyxeleditor でイメージ番号「2」にマップチップ画像をいっぱい作る
- pyxeleditor をマップ作成モードに変更し、マップ番号「0」にイメージ番号「2」のマップチップを配置していく
- 実際のゲーム画面に表示する

となります。

7.1 マップチップの作成

それでは pyxeleditor を起動して、マップチップを作成していきたいと思います。マップチップの最小単位は 8x8 ドットのチップになります。複数のチップをまとめて1つの画像にしても OK です。

まずは作りたい背景をイメージしてみます。せっかくの「アンパンマン」なの

第7章 背景を作成する

で、パン工場か、学校とかを舞台にしてみようかと思います。とりあえず、作りやすそうな学校の「校庭」をイメージしてマップチップを作ってみたいと思います。

いつものように Visual Studio Code の上部メニューの「Terminal」→「New Terminal」をクリックしてターミナルウインドウを開き、「pyxeleditor mychara」を入力して ENTER します。「pyxeleditor」が起動したらイメージ画像番号を「2」に変更します。

校庭ベースチップを作成

とりあえず、校庭=砂をイメージして、色番号=7 のみの 8x8 マップチップを左上に1枚作成します。

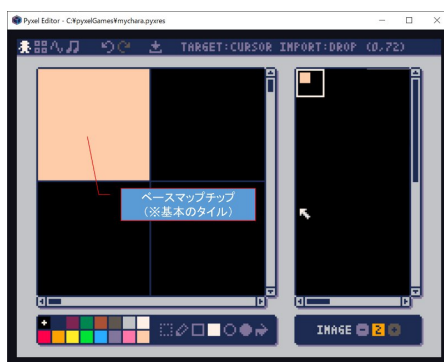


図 7.1: ベースマップチップ

基本はこのマップチップをデフォルト（マップすべてに配置）とし、それ以外のマップチップをで色々な背景を実現していく、という感じにします。

トラックマップチップ作成

校庭といえば、白マーカーで線を引いた「かけっこ用」のトラックの模様がほしいです。複数のマップチップを使い、

-  左上用の丸角マップチップ

第7章 背景を作成する

-  上用の直線マップチップ
-  右上用の丸角マップチップ
-  左側用の直線マップチップ
-  右側用の直線マップチップ
-  左下用の丸角マップチップ
-  下用の直線マップチップ
-  右下用の丸角マップチップ

を作成します。（それぞれ 8x8 のマップチップ）イメージ画像番号「2」上では次図のようになります。

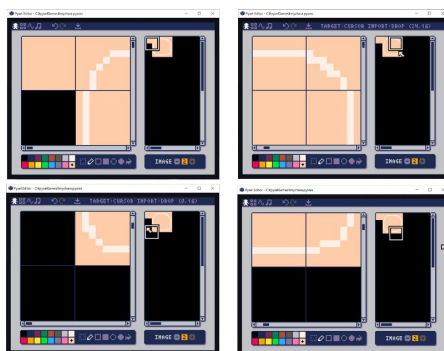


図 7.2: トラックマップチップ

7.2 マップデータの作成

ここまでできたら、さっそく実際のマップデータの作成を行ってみます。

pyxeleditor の上部メニューの「」ボタンをクリックしてマップ作成モードに切り替えます。すると以下のような画面に切り替わります。

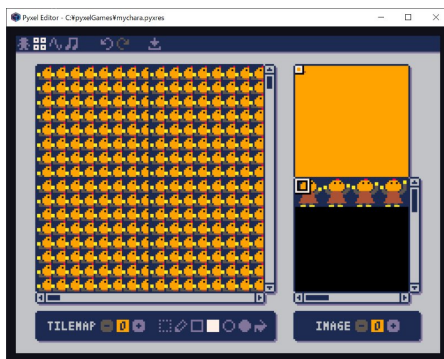


図 7.3: マップチップ切り替え①

最初はマップ番号 (TILEMAP) 「0」のマップにはイメージ画像番号「0」のチップが自動的に配置されてしまっております。アンパンマンの一部が表示されて気持ち悪いですね (笑) まずは、「マップ番号 (TILEMAP) 「0」のマップにはイメージ番号「2」のチップを使うよ」という設定を行います。pyxeleditor の画面右下部の「+」をクリックしてイメージ番号を「2」に切り替えます。



図 7.4: マップチップ切り替え②

イメージ番号「2」のマップチップを編集ツールを使って配置していきます。

第7章 背景を作成する

主に使う編集ツールは

-  : 選択したマップチップで1マス埋めます
-  : ドラッグ&ドロップした場所を選択したマップチップで埋めます
-  : 同一のマップチップエリアを選択したマップチップに変更します。

*   : 使い勝手が悪いので利用しません

今回の場合は、ベースのチップ以外はトラックの描写なので、「」ツールのみを使ってマップを作っていきます。

マップの作成方法は

- まず、右画面の配置したいマップチップ（1枚 or 複数）を選択
- 次に左画面でマップチップを配置したい位置にマウスでクリックすればマップチップが配置されますとなります。

まずは学校のトラックっぽい校庭を描いてみましょう。

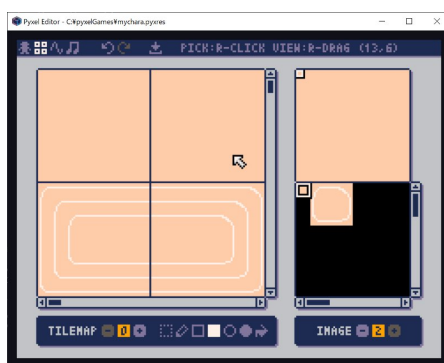


図 7.5: マップ作成画面で学校を描く①

第7章 背景を作成する

こんな感じでとりあえずトラック描写用に9枚のタイルデータあれば組み合わせで学校のトラックを描くことができます。

ついでに学校の校舎とかも用意してみたいと思います。pyxeleditor の画面上



部メニューの「」をクリックしてマップチップ (or キャラクター) 作成画面に移動します。校舎に使いそうな「壁」や「玄関 (昇降口)」、「窓」とかのマップチップを色々用意していきます。ついでに花壇とかも作ってみちゃいたいと思います。

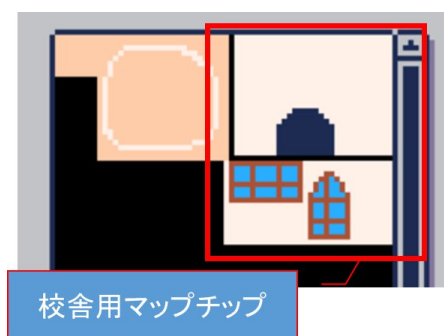


図 7.6: 校舎用マップチップを作る

マップチップができたら再度マップ作成画面に移動してマップチップを配置していきます。

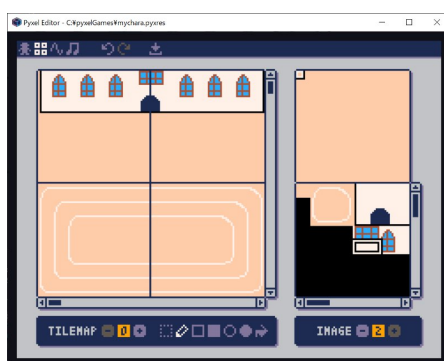


図 7.7: 校舎用マップチップを配置する

第7章 背景を作成する

なんかそれっぽくなりました。他にも「鉄棒」とか「砂場」とか色々足してみると面白いですね。



それでは最後に忘れずに「」ボタンをクリックして保存して pyxeleditor を終了します。

7.3 実際のゲームに組み込んでみる

実際のゲームに組み込む方法は簡単です。今回の場合は、128x128 ドットの画面を表示し続けるだけで、マップ画面は移動しないので以下の関数を追加して、「draw」関数内で使えば、それで完成です。

```
def drawMap(self):
    tm = 0
    w = int(WIDTH / 8)
    h = int(HEIGHT / 8)
    pyxel.bltn(0,0,tm,0, 0,w,h)
```

今回追加したい「drawMap」関数を少し説明します。マップデータを表示する「pyxel.bltn」のパラメータの意味は、

```
pyxel.bltn(
    マップを表示する左上の x 座標 (開始位置) ,
    マップを表示する左上の y 座標 (開始位置) ,
    マップ番号,
    指定したマップ番号の切り出し x 座標開始位置,
    指定したマップ番号の切り出し y 座標開始位置,
    切り出す横のタイル数,
    切り出す縦のタイル数)
```

となります。

ポイントは6番目と7番目のパラメータに「横のタイル数」「縦のタイル数」を指定する、ということです。マップデータのドットサイズではなく「タイル数」であることに注意してください。

第7章 背景を作成する

ゲーム画面が128x128だとすると、「pyxeditor」のマップ作成画面の1画面分なのでとってもわかりやすいです。(マップ作成画面は16x16 タイルなので、ドットサイズに変換すると16*8=128 ドットとなるわけです)

それでは、実際にゲーム画面を128x128にしてマップ表示を組み込んでみたいと思います。新規ファイル「sample07-01.py」を作成し、次のプログラムを記述してください。

リスト 7.1: sample07-01.py

```
1: import pyxel
2: import random
3: import pyxelChara as Chara
4:
5: WIDTH = 128
6: HEIGHT = 128
7:
8: class App:
9:     my_x = 0
10:    my_y = 0
11:    interval = 30
12:    cnt = 0
13:    enemyList = []
14:
15:    def __init__(self):
16:        pyxel.init(WIDTH, HEIGHT)
17:        pyxel.load('mychara.pyxres')
18:
19:        self.myChara = Chara.MyChara()
20:        pyxel.run(self.update, self.draw)
21:
22:    def makeEnemy(self):
23:        self.enemyList.append(Chara.Enemy(WIDTH, HEIGHT))
24:
25:    def checkTimer(self):
26:        bRet = False
27:        self.cnt = (self.cnt + 1) % self.interval
28:        if(self.cnt == 0):
29:            bRet = True
30:        return bRet
31:
32:    def update(self):
33:        self.my_x = pyxel.mouse_x
34:        self.my_y = pyxel.mouse_y
35:        if(self.checkTimer() == True):
36:            self.makeEnemy()
```

第7章 背景を作成する

```
37:
38:     for i in reversed(range(0, len(self.enemyList))):
39:         if(self.isCollision(i, self.my_x, self.my_y) == True):
40:
41:             self.myChara.startAnimation()
42:             self.enemyList[i].startAnimation()
43:
44:             if(self.enemyList[i].delete_flg == True):
45:                 del self.enemyList[i]
46:
47:
48:     def isCollision(self, _i, _x, _y):
49:         bRet = False
50:         my_x = _x + 8
51:         my_y = _y + 8
52:         if(self.enemyList[_i].ani_flg != True
53:            and self.enemyList[_i].delete_flg == False):
54:             if(
55:                 (self.enemyList[_i].x <= my_x)
56:                 and (self.enemyList[_i].x + 16 >= my_x)
57:                 and (self.enemyList[_i].y <= my_y)
58:                 and (self.enemyList[_i].y + 16 >= my_y)
59:             ):
60:                 bRet = True
61:         return bRet
62:
63:     def drawMap(self):
64:         tm = 0
65:         w = int(WIDTH / 8)
66:         h = int(HEIGHT / 8)
67:         pyxel.bltn(0,0,tm,0, 0,w,h)
68:
69:     def draw(self):
70:         pyxel.cls(7)
71:         self.drawMap()
72:         for i in reversed(range(0, len(self.enemyList))):
73:             x = self.enemyList[i].x
74:             y = self.enemyList[i].y
75:             if(self.enemyList[i].delete_flg == False):
76:                 self.enemyList[i].drawImage(x, y)
77:
78:         self.myChara.drawImage(self.my_x, self.my_y)
79: App()
```

ここで1つ注意することがあります。「drawMap」関数でマップ画面を表示さ

第7章 背景を作成する

せるタイミングは draw 関数内で「cls」関数で画面をクリアした直後がベストです。

```
71:     self.drawMap()
```

draw 関数が実行した順番が古いほど、背面になるため、背景画像であるマップ画面の描写は画面クリア直後がベストなのです。

それでは完成したゲームを実行してみてください。

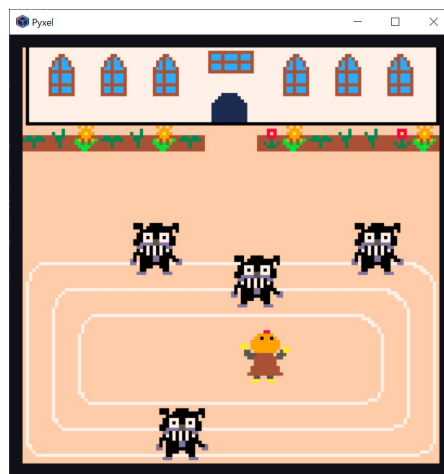


図 7.8: 完成したゲームで遊んでみる

いいですね！学校にいたずらしに来た「ばいきんまん」を「アンパンマン」がアンパンチするゲームが完成しました！

第 8 章

さいごに（おまけに）

8.1 モチーフ替え

最後にカバー詐欺と思われてしまうのは嫌なので「モチーフ替え」としてアンパンマンではなくドラクエ風世界観にするためにキャラクターをごっそり変更する改造の方法も学んでみたいと思います。まず、pyxeleditor を使い以下のリソースファイルを用意しました。用意したリソースファイル「quest.pyxres」は以下のようになっています。

- イメージ番号「0」に勇者（背中）のアニメーション



図 8.1: DQ 風イメージ（勇者）

- イメージ番号「1」に複数のDQ風モンスター

第8章 さいごに（おまけに）



図 8.2: DQ 風イメージ（モンスター）

このリソースファイルは以下サイト（GitHub）から入手可能です。

- DQ風もぐらたたき (Github)

https://github.com/tinycore-hisanori/pyxelGame_moguraDQ

既存の「pyxelChara.py」を以下のように修正することでランダムで出現する敵を変えて、モンスターを倒す、DQ風もぐらたたきの出来上がりです。

リスト 8.5: pyxelChara.py の MyChara クラス変数定義

```
9:      ani_max = 5
```

自キャラのアニメーション数を 6 → 5 に変更します。新しい画像に合わせて自キャラのアニメーションの最大値の変更が必要です。

リスト 8.5: pyxelChara.py の Enemy クラス変数定義

```
42:     ani_max = 5
```

敵キャラのアニメーション数を 8 → 5 に変更します。こちらも自キャラ同様に新しい画像に合わせて敵キャラのアニメーションの最大値の変更が必要です。敵キャラは複数登場させますが、すべて同じアニメーション数にしておくのが（プログラムが）楽かと思います。

リスト 8.5: pyxelChara.py の Enemy クラス変数定義

第8章 さいごに（おまけに）

```
47:     enm_no = 0
```

敵キャラ変更用の変数定義（追加）します。今回、敵キャラを4体用意したので、変数「enm_no」を使い敵キャラを切り替えできるようにします。

リスト 8.5: pyxelChara.py の Enemy クラス __init__ 関数内

```
52:     self.enm_no = random.randint(0, 3)
```

敵キャラの生成時にランダムでキャラ変更（追加）します。ランダムで選択されるモンスターは、

- 0:スライム風モンスター
- 1:ドラキー風モンスター
- 2:りんごおばげ
- 3:りゅうおう風モンスター

のどれかのキャラになります。

リスト 8.5: pyxelChara.py の Enemy クラス drawImage 関数内

```
73:     0 + (self.ani_no * 16), 0 + (self.enm_no * 16),
```

敵キャラの画面描写自にキャラ画像を変更できるようにします。イメージ番号「1」から切り出す開始Y座標が固定で「0」だったものを $0 + (\text{self.ani_no} * 16)$, $0 + (\text{self.enm_no} * 16)$, の計算式でキャラごとにY開始位置を変更できるようにしました。

myChara.py の改造ができれば sample07-01.py を実行してみてください。

第8章 さいごに（おまけに）

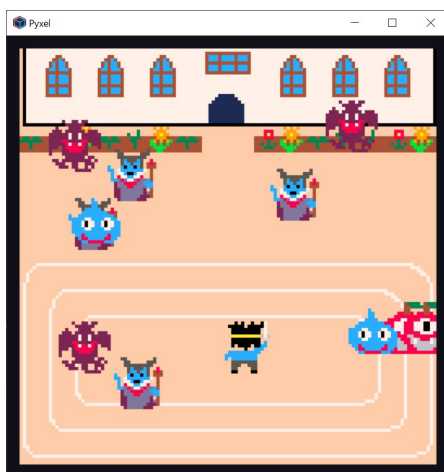


図 8.3: ゲームのモチーフ替え (DQ 風)

ドラクエ風ゲームにモチーフ替えすることができました。マップチップもいっそダンジョン風にしてもよいかもしれません。

8.2 その他にやるとよさそうなこと

ゲームができた！と喜ぶたいところですが、まだ色々改良の余地がたくさん残っているのが現状です。例えば

- ・ 敵を倒した数のスコアアタックを導入
- ・ ゲームに制限時間を設ける（時間内に何匹倒せたか、を競う）
- ・ 当たり判定をもっと精度よくする（矩形判定→円判定に変更する）
- ・ 敵キャラの動きをもっと複雑にする（動きまわったりさせる）
- ・ 逆に敵キャラからの攻撃をできるようにする
- ・ 音楽や効果音をつける

などがあります。ゲームエンジン「pyxel」の機能としてまだ紹介していない音楽とか効果音は pyxeleditor で実は作成できるのですが、これだけで1冊技術書ができてしまうくらいの内容なので、本誌では取り上げておりません。次の巻を出す機会があればそのときには取り上げたいと思います。

第8章 さいごに（おまけに）

ただ、せっかくなので、ゲームエンジン「pyxel」の Tips 的なまめ知識をいくつか紹介できればと思います。今回紹介する Tips は、

- pyxeleditor で作成したリソースファイルは分解可能
- python が無い環境でも実行できる実行ファイルを作成するの2つを紹介して終わりにしたいと思います。

8.3 Tip1：pyxeleditor で作成したリソースファイルは分解可能

pyxel でゲームを作っていると「あっ！前に作ったマップチップを別のリソースファイルでも使いたい」とか結構あったりします。pyxel のバージョンが 1.0 の頃はそれができなかったのですが、1.2 以降ではそんなことも可能になりました。pyxeleditor で作成する「pyxres」ファイルは実は「ZIP」ファイルなので、ZIP ファイルとして解凍ができます。試しに今回作成した「mychara.pyxres」の拡張子を ZIP に変更して解凍してみたいと思います。

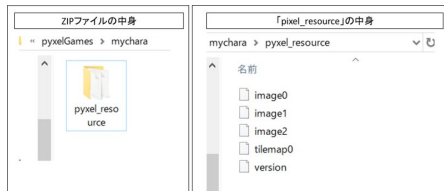


図 8.4: mychara.pyxres を分解してみる

このように「pyxel_resource」フォルダの下に

- image0
- image1
- image2
- image2
- tilemap0
- version

というファイルが格納されております。「version」ファイルには作成した pyxel

第8章 さいごに（おまけに）

のバージョン情報が入っており、おそらく今後 pyxel のバージョンアップの際に読み込み可能なリソースかどうかを判断するために使うのでしょう。

もし、他の「pyxres」ファイルと画像や音楽データを取り替えたい場合は、ZIP ファイルにしてファイルを上書きすれば簡単に取り換えが可能です。

8.4 Tip2：python がない環境でも実行できる実行ファイルを作成する

「pyxelpackager」というコマンドを使うと、python 環境がなくてもゲームができる実行ファイルを作ってくれるようです。使い方は

```
pyxelpackager python ファイル名
```

で必要なファイルをビルドし、WINDOWS ならば EXE（実行ファイル）を生成してくれます。

ただし、ビルドするためには「pyxres」ファイルを適切な場所に配置し、ソースも少し書き換える必要があります。

pyxres ファイルを「assets」フォルダに格納する

メインプログラムの python ファイルがある場所に「assets」というフォルダを作成し、リソースファイルはすべてその「assets」フォルダに入れる必要があります。pyxres ファイルを「assets」フォルダの中に移動します。ここでは「mychara.pyxres」を assets フォルダ内に移動します。

第8章 さいごに（おまけに）

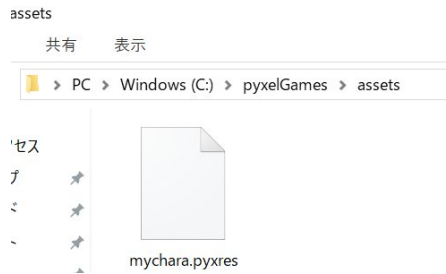


図 8.5: mychara.pyxres を移動する

プログラムを修正する

リソースファイルを移動したので、プログラムを以下のように修正します。

リスト 8.6: 修正前 sample06-01.py

```
pyxel.load('mychara.pyxres')
```

↓

リスト 8.7: 修正後 sample06-01.py

```
pyxel.load('assets/mychara.pyxres')
```

試しに python のままで実行可能な確認してみてください。ちゃんと pyxres ファイルが読み込まれてゲームができていれば OK です。

実行ファイルを作成する

それでは、「sample06-01.py」を実行形式に変換してみたいと思います。

```
pyxelpackager sample06-01.py
```

第8章 さいごに（おまけに）

マシンによってはしばらく待つかもしれません。私の Core i5 マシンでは30秒～1分くらいでビルドできました。

ビルドに成功したら次図のように「dist」フォルダに実行ファイルが生成されております。

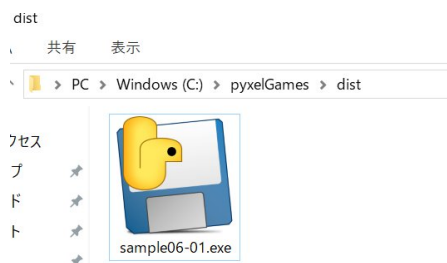


図 8.6: 実行ファイル生成

では、さっそく実行してみてください。python のときと同じようにゲームが動作すれば成功です。この実行ファイル1つあればどこでもゲームを遊ぶことができます。

8.5 まとめ

今回、初心者向けとして python のゲームエンジン「pyxel」を使ったゲーム開発を解説させていただきました。私自身、IT 専門学校の講師をさせていただいており、その中で「pyxel」を授業で実際に使ってプログラム学習教材にしております。

pyxel のよいところは「できることが限られていること」にあると思います。pyxel の「できることが限られている」というのは一見、デメリットのように感じるかもしれませんが、覚えることが少なくゲームを作ることができる、というのは学習のハードルを一気に下げる効果があり、途中で萎えることなくゲームを完成させることができる（かもしれません）

pyxel は SDL2 というマルチプラットフォームで動作可能な描写ライブラリを使っております。今後、pyxel プログラムを iOS や Android に変換するといっ

第8章 さいごに（おまけに）

たことも可能になるかもしれません。（2019/9/14 の現時点では未対応）

とっても簡単にレトロゲームが作れる面白いゲームエンジンなので、これを機にぜひゲーム作りにチャレンジしてみてください！

ゲーム開発初心者のための HowTo pyxel でサクッとレトロゲーム開発！

2019 年 9 月 17 日 初版第 1 刷 発行

著 者 Hisa Mail: hisanori.andou@tinycore.co.jp

発行所 タイニーコア（株） <https://tinycore.co.jp>

印刷所 日光企画

(C) 2019 TinyCore Co., Ltd.